Generative machine learning approaches to optimization

Victor Alves and John R. Kitchin*

Department of Chemical Engineering, Carnegie Mellon University, Pittsburgh, PA 15213

E-mail: jkitchin@andrew.cmu.edu

Abstract

We present a generative machine learning approach to optimization problems. The key idea is that generative models can learn the joint probability distribution of variables and their derivatives, or transform simple distributions to target distributions. By conditioning on desired properties (e.g., derivatives equal to zero), solutions are obtained by sampling. We demonstrate this using Gaussian mixture models, which approximate joint distributions, and conditional flow matching, which learns distribution transformations. Examples include root finding, unconstrained and constrained optimization, parameter estimation, and state mapping. This pedagogical work shows that generative models offer a fundamentally different approach to optimization, one that is data-driven, does not require iterative solvers, and naturally handles problems with multiple solutions by representing the full solution distribution. We discuss limitations and conclude the approach shows promise as an alternative to conventional methods.

1 Introduction

The field of optimization is immense, with elegant solutions tailored to specific problem applications, depending on the nature of the objective function and constraints that are to be

solved (e.g., linear *vs.* nonlinear, convex *vs.* nonconvex, differentiable *vs.* nondifferentiable), the nature of the variables involved in the optimization problem (discrete, continuous, or both), and lastly, the nature of the solution itself, global *vs.* local optimization. This myriad of problems that arise in many fields of science and engineering has led to the development of numerous solution techniques, ranging from the initial work of linear programming and the Simplex method by Dantzig,[1] to the advent of robust nonlinear programming techniques including sequential quadratic programming (SQP),[2–4] and the popularization of interior-point methods.[5] Additionally, solutions of mixed integer nonlinear programming (MINLP) algorithms[6] and recent innovations on generalized disjunctive programming (GDP) which embeds logical decisions into mathematical programming formulations,[7] and global optimization[8–10] are all examples of how rich and deep the literature on the topic is. Among popular programming languages implementations of such algorithms, in Python the `scipy` library is particularly notable,[11] as well as the `Pyomo`[12] framework, GEKKO,[13] Gurobi,[14] Mosek,[15] and in Julia there is JuMP.[16] These libraries include both algebraic optimization modeling languages and commercial and/or open-source optimization solvers. Despite all the work aforementioned, the list is by no means exhaustive, and many other interesting research directions are currently available and being actively developed.

We see a particular interest in algorithmic development of solutions to optimization problems that are specific to the nature of the problem, its variables and local/optimal convergence criteria. However, less attention has been given to the use of emerging and powerful tools such as generative machine learning (genML) in the well-developed area of optimization. It is clear that optimization/mathematical programming is one of the cornerstones of machine learning (independent of the generative nature or lack thereof), but the reverse application, that is, genML as a tool to solve optimization problems, has been less unexplored.

To orient the reader on where a generative-based optimization approach fits in the optimization landscape, we illustrate genML-optimization in Figure 1, based on the classification

of optimization problems in Ref. 17. We intentionally modified the original flowchart in Refs. 17 and 18 to show which problems we are able to currently employ genML-optimization on. Although promising, there is still a lot to cover when compared to the current state-of-the-art in optimization.

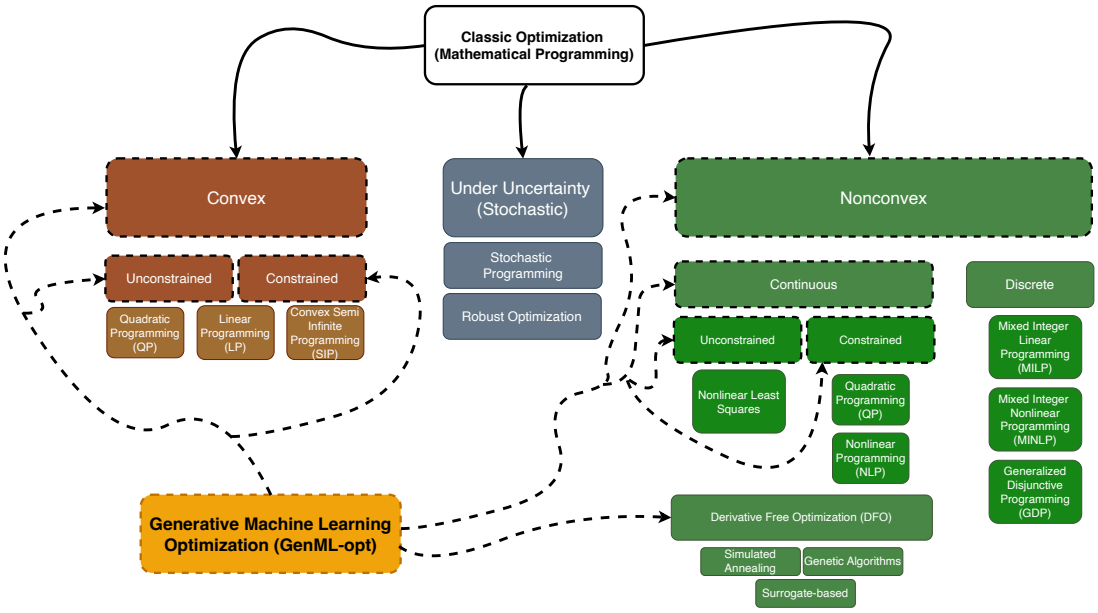


Figure 1: Tree containing classes of optimization problems, based on flowcharts available in the literature.[17,18] We show with connected dashed lines and also highlighting classes of optimization problems that we were able to currently solve using generative machine learning applied to optimization - genML-opt.

In this manuscript we explore a different way to think about optimization through generative machine learning (genML).[19] The basic concept in genML is that if one knows the joint probability distribution of a set of variables, then it is possible to generate samples of that distribution. Alternatively, if one knows how to transform a reference distribution to the joint probability distribution of those variables, then one samples the reference distribution and transforms it to a sample of the desired distribution.

In fact, generative models have been applied to optimization problems before. Invertible neural networks are a special class of models that are used to model the joint distribution of inputs and outputs.[20] Once they are trained, feasible solutions can be found by sampling them. Diffusion models have been applied to inverse problems in mechanics.[21] We

recently applied genML to solve an inverse linear problem.[22] Generative models can also be used to enhance combinatorial optimization problems[23] where they can be used to generate near-optimal solutions that are subsequently refined by conventional algorithms. There are applications of generative machine learning models to dynamic systems forecasting[24] and for generating potential pre-screened solutions of multi-objective mathematical programs.[25] Finally, generative adversarial models have been used in optimization.[26]

Probably the most common genML method used today is in the generation of text[27] and images.[28] Large language models (LLMs) are a form of genML where the most likely sequence of words to follow a prompt is generated. LLMs are not the focus of this work. In image generation, a genML model is trained on a large set of images to "learn the image distribution", and then images are generated by sampling that distribution. We find it helpful to think of an image first as a matrix of pixel values (e.g. the RGB intensities), and then to think of that matrix as a flattened vector. Then, a large set of images will have a distribution of values in that vector space where there are correlations (covariance) between the elements of the vector that define an image space. Thus, one generates a sample vector, and then reshapes it into a matrix, which can be visualized as an image. This is the motivation for the present work.

This approach may not seem immediately helpful in solving optimization problems until we make these observations:

1. **Instead of a collection of pixel values, the vector elements can be inputs and outputs of a model or system.** By concatenating those we obtain a vector analogous to the flattened image vector. The vector elements can include derivatives of the model, or other easily derived features if desired. If there are many such vectors, then they will form a distribution in that vector space where there are correlations (covariance) between the elements, e.g. the inputs and outputs. Since it is evident generative models can learn that distribution for images, it seems reasonable they could learn this distribution for inputs/outputs, which we demonstrate in this work.

Thus, if we can learn or estimate the joint probability distribution, we can generate samples of inputs and outputs from it.

2. **Generation of these vectors can be conditional.** That is, we can specify certain values of the variables, update the joint probability distribution of the remaining variables, and then generate those variables consistent with a desired condition. Thus, if we condition on inputs, we can then generate samples of the output, and vice versa. This is directly related to root finding; we might condition on the output being zero, and generate the inputs that lead to that result.

To connect such ideas to the main scope of this work (e.g., finding minima/maxima) it is necessary to obtain the connection between inputs of a model and its corresponding derivatives. If we know that joint probability distribution, then we generate samples conditioned on the derivatives being equal to zero for example (e.g., unconstrained optimization case). Furthermore, it is also possible to include constraints by leveraging a Lagrangian function, or any other augmented function, and condition the derivatives of this augmented function to be zero. Lastly, other cases can also be tackled, such as parameter estimation, in which we need the joint probability distribution between the parameters and an observed output(s). In this case, to estimate parameters from an observation, we simply condition on the observation to find the parameters. These are all related to solving inverse problems in optimization.

An alternative approach to generative modeling is to transform a simple distribution into a more complex distribution. In diffusion and flow-matching models, this transformation is learned from data, and then it is used conditionally to generate samples of a target distribution (e.g. the outputs) from samples of a simpler distribution such as a normal distribution.

The Bayesian statistical approach to inverse problems, as described by Waqar et al.,[29] shares conceptual similarities with the generative optimization framework presented here. Both approaches address ill-posed inverse problems where solutions may not exist, may not

be unique, or may be sensitive to noise, by producing probabilistic solutions that quantify uncertainty. In Bayesian statistical inversion (BSI), prior beliefs about parameters are combined with observed data through Bayes' theorem to obtain posterior distributions, which are then sampled using Markov Chain Monte Carlo (MCMC) methods. In contrast, our generative approach learns the joint distribution directly from data using Gaussian mixture models or flow matching, then samples solutions by conditioning on desired output values. While BSI requires explicit specification of prior distributions and likelihood functions, generative models learn these relationships implicitly from training data. Both approaches naturally handle problems with multiple solutions by representing the full solution distribution rather than returning a single point estimate. The key practical difference is that BSI is model-based (requiring a forward model to evaluate likelihoods), while our generative approach can be purely data-driven when sufficient training data is available.

We argue that this approach is fundamentally different from a conventional view point in the mathematical programming and engineering research communities. In our proposed work, there may be a model involved, however this is not strictly necessary; the approach can be completely data-driven in some cases as previously stated. This feature can be particularly useful in our own field (e.g., chemical engineering) in which sensor data can be available, but a first principles model may not be fully developed. In this case, the joint probability distribution among sensor data may still be inferred by employing our proposed approach and the optimal solution landscape can be pre-screened while a fundamental/first principles model is being developed. Furthermore, if a rigorous model based on first principles is developed, our approach can help generate potentially feasible solutions that can be employed in a rigorous mathematical program.

Additionally, we do not need to think about a forward or inverse model as is typically done in engineering applications. Instead, the joint probability distribution (or the transformation function) is simultaneously both of these, and conditioning determines what direction that one may desire, even if it is a "mixed mapping direction" (e.g., it is entirely possible to condi-

tion on some inputs and some outputs). The practical implication of the proposed approach is that one is able to generate solutions to optimization problems without using conventional approaches such as Newton-Raphson iterative methods, or derivative-free simplex methods such as Nelder–Mead.[30]

Summarizing, genML has the potential to serve as an alternative optimization solution approach to several classes of problems listed above, including linear and nonlinear programming (both constrained and unconstrained), parameter estimation problems, inverse mapping, and root finding. Intriguingly, we have observed that the solution approach to these types of problems is remarkably consistent, as opposed to highly-tailored and specific algorithms (the current *status quo*) as long as data is available.

We do not anticipate that generative machine learning formulations will replace classic mathematical programming development. Instead, we believe that a branch of optimization problems that can be solved by using genML will employ even more sophisticated optimization algorithms at the training and inference of generative models. In short, the optimization layer will always be present, and its interaction with the researcher/practitioner will be what might change within the next years.

Lastly, it is essential to note that we make no claims of superiority of generative methods over the well-established mathematical programming-based solution methods; indeed, quite the contrary. Conventional algorithms offer many advantages to the ideas presented here. Instead, we focus on developing a proof of concept for a new way to think about solving these problems. We do this through a series of examples that span root finding, minimization including constraints, parameter estimation, and state mapping, as shown in Section 3. The remainder of this paper is structured as follows: in Section 2 the methods employed in this work are presented, including an introduction of Gaussian mixture models and conditional normalizing flow matching techniques. In Section 3 several case studies are presented to show the potential of the ideas here presented. Lastly, in Section 4 we discuss final conclusions and bring ideas about potential future work on this field, as we anticipate that it is an emerging

one.

2 Methods

2.1 Gaussian mixture models

Gaussian Mixture Models (GMMs) are a powerful and versatile mathematical tool that is able to characterize complex and multimodal distributions by combining multiple individual Gaussian functions. They are mostly used in machine learning applications related but not limited to data clustering,[31,32] as well as being recognized in the literature for pattern recognition in machine learning[33] applications. Additionally, in the field of robotics, Gaussian Mixture Regression (GMR) has been used as a generative modeling framework,[34] and it is considered a generative model[35] itself. We follow the notation from Ref. 34 for the mathematical definition of GMMs. However, the reader should note that equally valid formulations are present in the literature.[33,35]

From a machine learning standpoint, the main task is, given a pair of vectors x and y, to learn a probability distribution as defined as in Eq. 1, which is the definition of a Gaussian mixture model.

$$p(\boldsymbol{x}, \boldsymbol{y}) = \sum_{k=1}^{K} \pi_k \mathcal{N}_k \left(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k}, \boldsymbol{\Sigma}_k\right) \tag{1}$$

In which $\mathcal{N}_k \left(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k}, \boldsymbol{\Sigma}_k\right)$ is the k^{th} Gaussian distribution in a mixture, defined by its mean $\boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k}$ and covariance $\boldsymbol{\Sigma}_k$. $\pi_k \in [0, 1]$ are defined as *weights* or *priors*, which quantify the contribution of each Gaussian to the mixture model.

2.1.1 Training

The training step corresponds to obtaining an accurate prediction of this unknown, probably multimodal and multivariable probability distribution, given a dataset defined by the

pair of vectors x, y. For simplicity, we can lump the parameters related to the formulation of Eq. 1 which includes the means, covariances and mixture weights defined as $\theta := \left\{\pi_k, \boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k}, \boldsymbol{\Sigma}_k : k = 1, \ldots, K\right\}$.

As a figure of merit is needed to fit this generative machine learning model, we resort to a likelihood formulation, defined as in Eq. 2.

$$p(x, y \mid \boldsymbol{\theta}) = \prod_{n=1}^{N} p\left(x_n, y_n \mid \boldsymbol{\theta}\right), \quad p\left(x_n, y_n \mid \boldsymbol{\theta}\right) = \sum_{k=1}^{K} \pi_k \mathcal{N}\left(x_n, y_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\right) \tag{2}$$

The log-likelihood[35] is formulated in Eq. 3. Ideally, one could solve for $\mathrm{d}\mathcal{L}/\mathrm{d}\boldsymbol{\theta} = 0$ and the optimal parameters θ^* that maximizes the likelihood would be obtained analytically. However, since Eq. 3 is not a closed-form expression/solution,[35] an iterative method is used. Expectation Maximization (EM) is applied in GMMs successfully and this is what packages such as `gmr`[34] and the scikit-learn[36] implementation of GMM does.

$$\log p(\boldsymbol{x}, \boldsymbol{y} \mid \boldsymbol{\theta}) = \sum_{n=1}^{N} \log p\left(\boldsymbol{x}_n, \boldsymbol{y}_n \mid \boldsymbol{\theta}\right) = \underbrace{\sum_{n=1}^{N} \log \sum_{k=1}^{K} \pi_k \mathcal{N}\left(\boldsymbol{x}_n, \boldsymbol{y}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\right)}_{=:\mathcal{L}} \tag{3}$$

2.1.2 Prediction

Following the conditional distribution property of each individual Gaussian, one is able to compute the conditional distribution $p(y \mid x, \theta)$ by first writing down the conditional distribution for a given k^{th} element.[34]

$$\boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k} = \begin{bmatrix} \boldsymbol{\mu}_{\boldsymbol{x}k} \\ \boldsymbol{\mu}_{\boldsymbol{y}k} \end{bmatrix}, \quad \boldsymbol{\Sigma}_k = \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k} & \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{y}_k} \\ \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{x}_k} & \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{y}_k} \end{bmatrix} \tag{4}$$

The conditional distribution of $\boldsymbol{y}$ given $\boldsymbol{x}$ for the k^{th} Gaussian component is defined as in Eq. 5:

$$\begin{aligned}\boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)} &= \boldsymbol{\mu}_{\boldsymbol{y}k} + \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{x}_k}\boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_{\boldsymbol{x}k}) \\ \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)} &= \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{y}_k} - \boldsymbol{\Sigma}_{\boldsymbol{y}\boldsymbol{x}_k}\boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k}^{-1}\boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{y}_k}\end{aligned} \tag{5}$$

To obtain the full conditional distribution $p(\boldsymbol{y} \mid \boldsymbol{x})$, we combine the component-wise conditional Gaussians using the posterior weights $\pi_{k|\boldsymbol{x}}$, which represent the probability of component k given the observed $\boldsymbol{x}$:

$$\pi_{k|\boldsymbol{x}} = \frac{\pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_{\boldsymbol{x}k}, \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_k})}{\sum_{l=1}^{K} \pi_l \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_{\boldsymbol{x}l}, \boldsymbol{\Sigma}_{\boldsymbol{x}\boldsymbol{x}_l})} \tag{6}$$

Finally, the final predicted distribution corresponds to a mixture of conditional Gaussians:

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \sum_{k=1}^{K} \pi_{k|\boldsymbol{x}} \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}, \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}) \tag{7}$$

If one desires to have them in a more compact form, we refer to our previous definition of the lumped hyperparameters as θ as

$$\theta := \left\{\pi_k, \boldsymbol{\mu}_{\boldsymbol{x}\boldsymbol{y}_k}, \boldsymbol{\Sigma}_k\right\}_{k=1}^{K} \tag{8}$$

which yields a predictive distribution form as in Eq. 9.

$$p(\boldsymbol{y} \mid \boldsymbol{x}; \theta) = \sum_{k=1}^{K} \pi_{k|\boldsymbol{x}}(\theta) \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{\mu}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}, \boldsymbol{\Sigma}_{\boldsymbol{y}|\boldsymbol{x}}^{(k)}) \tag{9}$$

Eq. 9 plays a major role in our work. As we draw samples based on conditioning, we can simultaneously infer variables that belong to either the x or y spaces, usually perceived as inputs and outputs, respectively. We show this in several applications related to optimization.

Lastly, we note that typical optimization techniques are used, more specifically during the training step of the generative approach, namely the expectation maximization (EM) algorithm.[37] However, this optimization step, which costs $\mathcal{O}(NK\mathcal{D}^3)$ per iteration for N samples, K mixture components and dimensionality $\mathcal{D}$, is performed offline and separately

from the applications in which the GMM is employed.

2.2 Conditional normalizing flow matching

Flow matching is a simulation-free approach to training continuous normalizing flows.[38] Unlike traditional normalizing flows that require computing expensive Jacobian determinants, flow matching learns a time-dependent vector field $v_t(x)$ that transforms samples from a simple base distribution p_0 (typically $\mathcal{N}(0, I)$) to a target data distribution p_1 through ordinary differential equations (ODEs).

Given a data sample $x_1 \sim p_1(x)$ and a base sample $x_0 \sim p_0(x)$, we construct a probability path $p_t(x)$ for $t \in [0, 1]$ that interpolates between the two distributions. A common choice is the conditional Gaussian path:

$$p_t(x|x_1) = \mathcal{N}(x|tx_1, (1 - (1 - \sigma)t)^2 I) \tag{10}$$

where σ is a small constant (typically 10^{-5}) to ensure numerical stability.

The corresponding conditional vector field that generates this path is:

$$u_t(x|x_1) = \frac{x_1 - (1 - \sigma)x}{1 - (1 - \sigma)t} \tag{11}$$

For optimization problems, we extend flow matching to learn conditional distributions $p(x|c)$, where x represents the decision variables and c represents the conditioning information (e.g., constraint values, target objectives, or system parameters).[39] This allows us to sample from different conditional distributions by simply changing the conditioning value.

The conditional flow matching objective is:

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t, p_1(x_1, c), p_0(x_0)} \left[\| v_\theta(x_t, c, t) - u_t(x_t|x_1) \|^2 \right] \tag{12}$$

where $v_\theta(x_t, c, t)$ is the learned vector field parameterized by a neural network with parame-

ters θ, $t \sim \text{Uniform}(0,1)$ is the time variable, $x_t \sim p_t(x|x_1)$ is a sample along the flow path, and (x_1, c) are paired samples from the training data.

We parameterize the vector field $v_\theta(x, c, t)$ using a multi-layer neural network with the following architecture: the Input (concatenation of $[x, c, t]$), the hidden layers (3 fully-connected layers with 64 units each and sigmoid activation functions), and the output (a vector field prediction $v \in \mathbb{R}^{d_x}$).

The choice of sigmoid activations (rather than ReLU) provides smooth gradients throughout the network, which is beneficial for learning continuous vector fields.[40] The model is trained using the following procedure. First, for each training sample, we store the input-output pair (x, c) where x represents the solution and c represents the conditioning information. During training, each mini-batch involves sampling data pairs (x_1, c) from the training set, sampling base noise $x_0 \sim \mathcal{N}(0, I)$, and sampling time $t \sim \text{Uniform}(0,1)$. We then construct the intermediate state $x_t = tx_1 + (1-(1-\sigma)t)x_0$ and compute the target vector field $u_t(x_t|x_1) = \frac{x_1-(1-\sigma)x_t}{1-(1-\sigma)t}$. The training objective minimizes the mean squared error between the predicted and target vector fields using the Adam optimizer[41] with learning rate 10^{-3}. To improve training stability, all input features are standardized to have zero mean and unit variance before training. All flow matching models are implemented in PyTorch.[42]

Training typically requires 500 epochs with batch size 64 for the problems considered in this work. To generate samples from the learned conditional distribution $p(x|c)$, we solve the ODE:

$$\frac{dx}{dt} = v_\theta(x, c, t), \quad x(0) \sim \mathcal{N}(0, I) \tag{13}$$

from $t = 0$ to $t = 1$ using the Euler method with $N = 100$ steps:

$$x_{t+\Delta t} = x_t + v_\theta(x_t, c, t) \cdot \Delta t, \quad \Delta t = \frac{1}{N} \tag{14}$$

The final state $x(1)$ represents a sample from $p(x|c)$. To obtain multiple samples, we simply repeat this process with different initial noise samples $x(0)$.

Conditional flow matching offers several advantages for optimization problems. First, unlike score-based models[43] or traditional normalizing flows,[44] flow matching can provide simulation-free training that does not require computing expensive score functions or Jacobian determinants during training. Second, the learned distribution naturally captures multi-modal solutions, enabling exploration of the entire solution space when constraints admit multiple feasible points. Third, the flexible conditioning mechanism allows rapid sampling of solutions for different problem instances by simply changing the conditioning value c without retraining the model. Fourth, the distribution of samples provides natural uncertainty quantification for the solutions. For problems with missing data or regions of low sampling density, flow matching learns to interpolate smoothly through these regions by leveraging the continuous nature of the learned vector field.

2.3 LLM usage in this work

We used Claude Code with the Opus 4.5 model extensively to develop the code in this work and for literature review. The manuscript was largely written by the authors. The GMM code used in this work was originally written by the authors, and then integrated into the work by Claude Code. The conditional normalizing flow code and the narrative describing it in the manuscript was created with Claude Code.

We note this here for two reasons. First is transparency. Second, we want to highlight how LLM usage enhanced and expanded the work. The first version of this work focused solely on the GMM approach with simpler examples. The use of an LLM enabled us to significantly expand the scope to include flow matching, and more sophisticated examples and visualization. The LLM also offers a new opportunity to engage with the work by "chatting" with the repository. We provide a `SKILL.md` file in the supporting repository that provides "Expert guidance for solving optimization problems using generative models (GMM and Flow Matching). Use when users need to solve optimization, inverse problems, or find feasible solutions under constraints using probabilistic sampling approaches."

3 Results and discussion

3.1 Basic concepts of generative modeling

In generative modeling there are mainly two options to build data-driven models. The first is to develop a model of the joint probability distribution between inputs x and outputs y, and conditionally generate samples, e.g. at fixed x or fixed y by updating the joint probability to reflect the residual degrees of freedom. The Gaussian Mixture Model is a generative model that falls into this category, in which analytical formulas to update the probabilities are readily available. We illustrate this conceptually below in Figure 2.

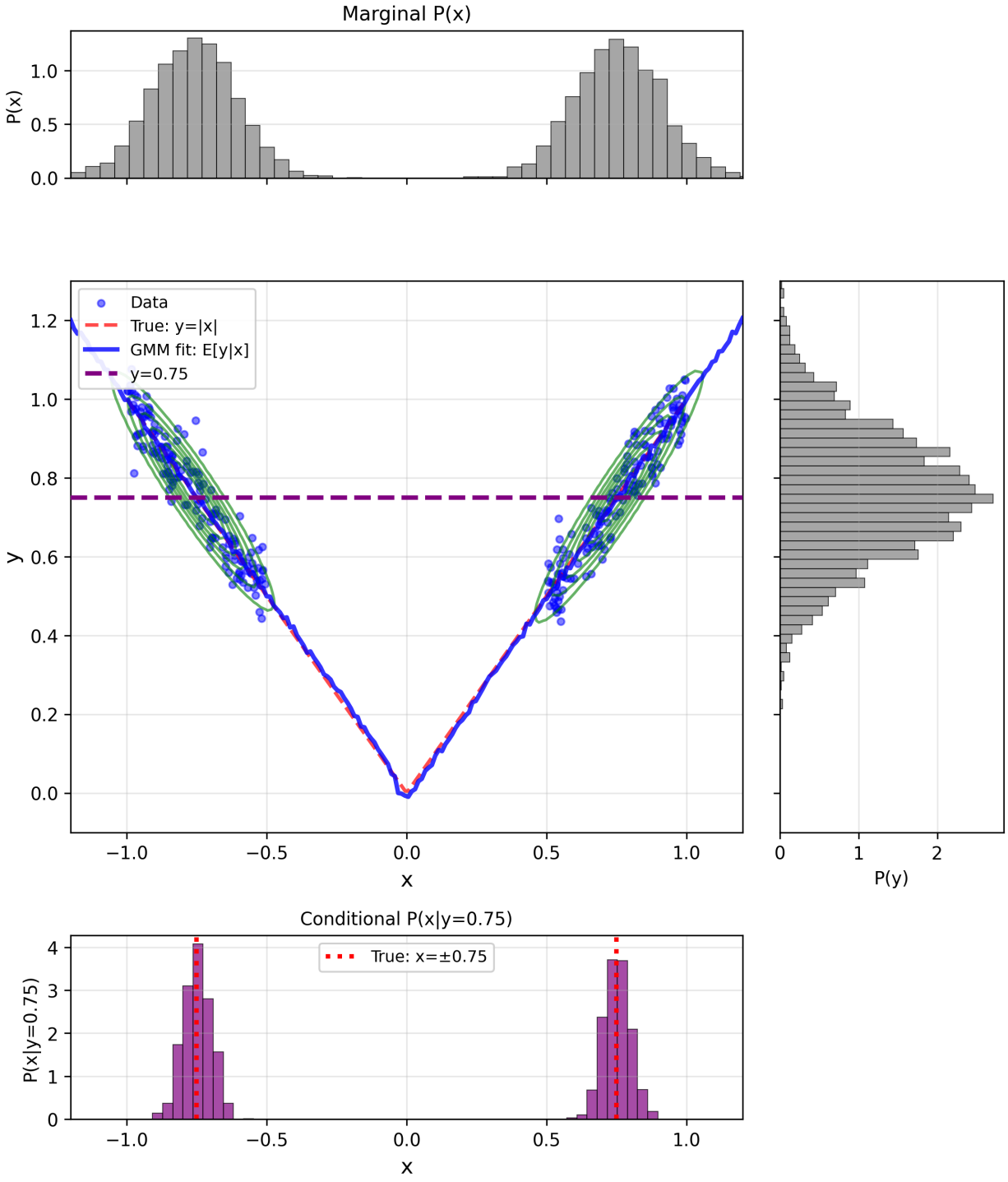


Figure 2: GMM fit for $y = |x|$ with missing data in the region $-0.5 \leq x \leq 0.5$. The central panel shows the data (blue points), true function (red dashed), GMM fit $E[y|x]$ (blue solid line), and GMM density contours (green). The marginal distributions $P(x)$ (top) and $P(y)$ (right) show the overall data distribution. The bottom panel shows the conditional distribution $P(x|y = 0.75)$, which is bimodal with modes at $x \approx \pm 0.75$, demonstrating the GMM's ability to capture multi-modal inverse solutions despite missing data.

The second approach corresponds to the use of conditional flow matching techniques. Instead of updating the joint probability distributions these models learn a conditional velocity field that transforms one distribution (often a simple normal distribution) into the target distribution. We show a result for this below in Figure 3 for the same problem we solved above with the GMM.

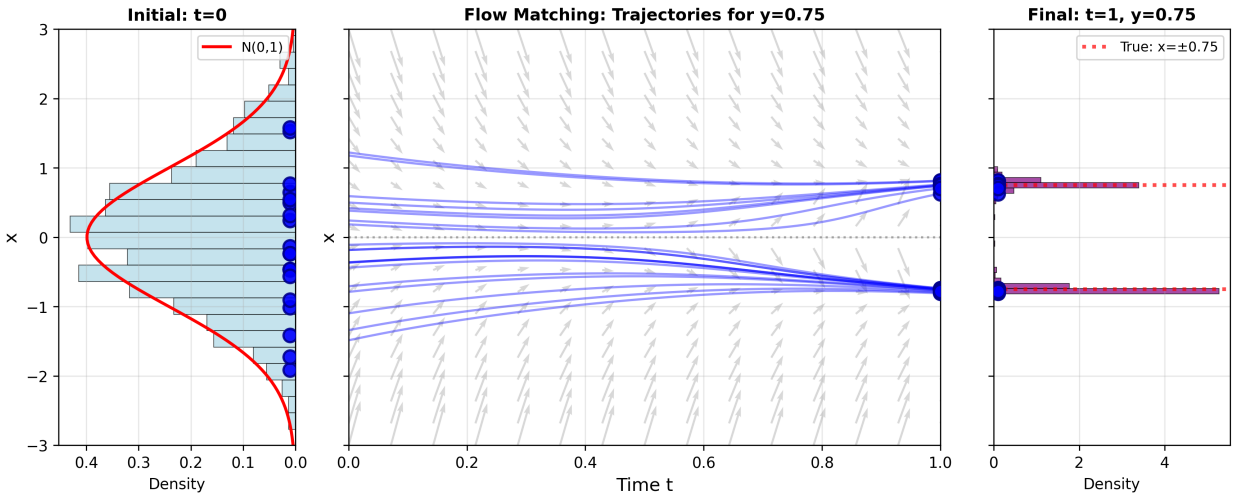


Figure 3: Flow Matching trajectories for $y = |x|$ with missing data. Left panel shows the initial noise distribution $N(0, 1)$ at $t = 0$. Center panel shows 20 trajectories evolving from noise to data, conditioned on $y = 0.75$, with gray arrows indicating the learned velocity field $v(x, t, y = 0.75)$. Right panel shows the final conditional distribution $P(x|y = 0.75)$ at $t = 1$, which is bimodal with modes at $x \approx \pm 0.75$, matching the true inverse solutions. Flow Matching learns to transform a simple Gaussian into a complex bimodal distribution conditioned on the target value.

In the next subsections we illustrate several examples related to chemical engineering that can employ the ideas proposed in this work.

3.2 Root finding

In root finding we seek values of x where $f(x) = 0$, including both scalar and vector-valued functions and input vectors. Traditional approaches include bisection (scalar functions), Newton-Raphson and related methods (both scalar and vector-value functions). Here we demonstrate a generative approach in which we learn the joint distribution $p(x, f(x))$ and

then condition on $f(x) = 0$ to generate samples of the roots.

A practically relevant example is finding compressibility factors from the Peng-Robinson equation of state (PR-EOS), widely used in chemical engineering for vapor-liquid equilibrium calculations. In compressibility factor form, the PR-EOS is a cubic equation as shown in Eq. 15.

$$Z^3 - (1 - B)Z^2 + (A - 3B^2 - 2B)Z - (AB - B^2 - B^3) = 0 \tag{15}$$

where $A = aP/(R^2T^2)$ and $B = bP/(RT)$ are dimensionless parameters computed from the substance's critical properties. In the two-phase region, this cubic has three real roots: the liquid compressibility factor Z_L, an unstable intermediate root, and the vapor compressibility factor Z_V. For propane at $T = 300$ K and $P = 10$ bar (in the two-phase region), we generate training data by evaluating the cubic residual $r(Z) = Z^3 - (1 - B)Z^2 + (A - 3B^2 - 2B)Z - (AB - B^2 - B^3)$ over the reasonable domain $Z \in [0.01, 0.99]$. We then fit generative models to the joint data $[Z, r(Z)]$. To find the roots, we condition on $r(Z) = 0$ and sample from the resulting conditional distribution $p(Z \mid r = 0)$. Figure 4 shows the conditional probability density, which exhibits peaks corresponding to the liquid ($Z_L \approx 0.035$), unstable ($Z \approx 0.13$), and vapor ($Z_V \approx 0.81$) roots. Furthermore, cluster analysis of the samples yields the expected results as shown in Table 1.

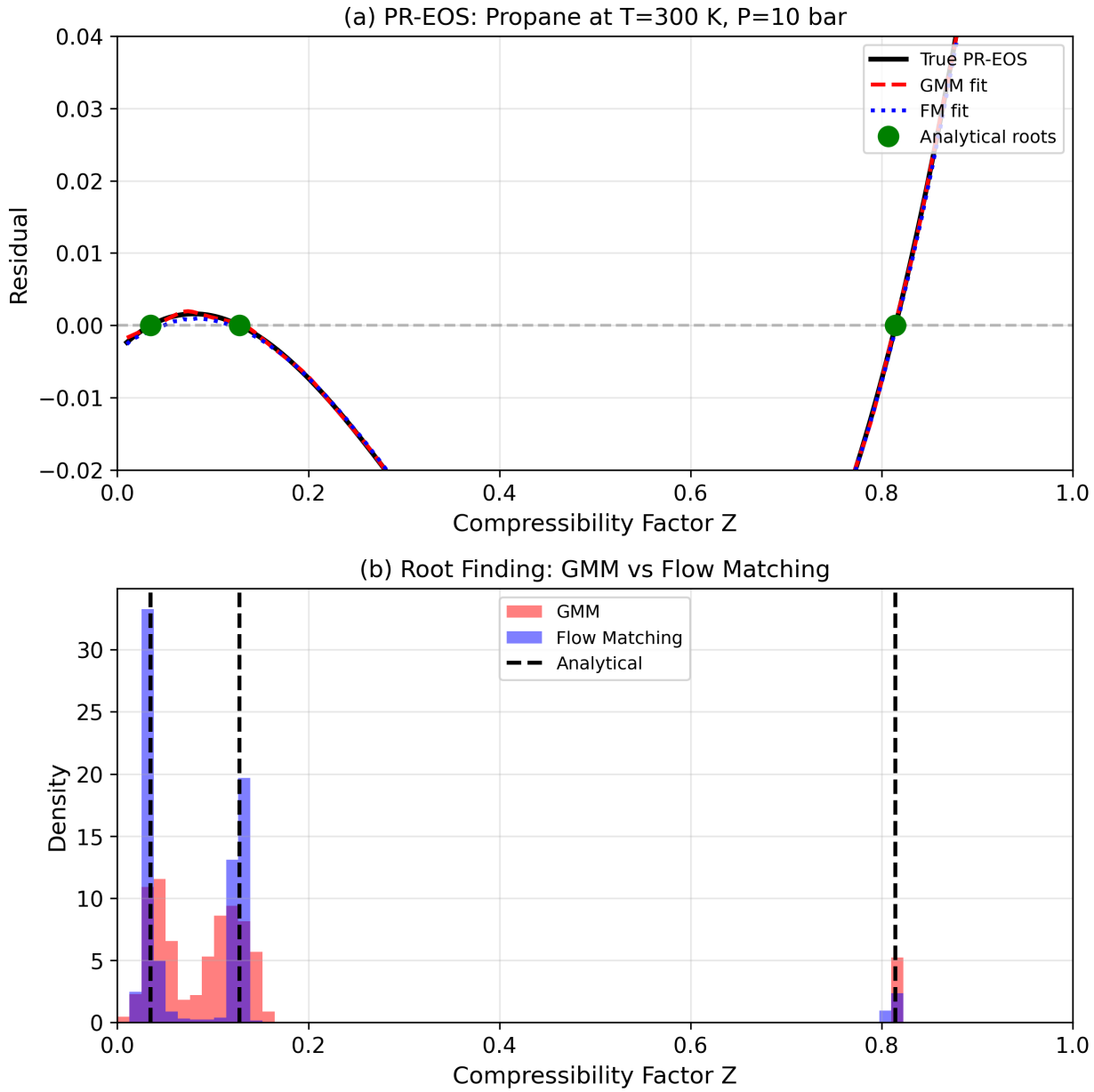


Figure 4: Comparison of Gaussian Mixture Model (GMM) and Flow Matching (FM) approaches for root finding in the Peng-Robinson equation of state. (a) The PR-EOS cubic residual for propane at T = 300 K and P = 10 bar, showing the true function (black solid line) along with the learned fits from GMM (red dashed) and FM (blue dotted). Both generative models accurately capture the functional relationship between compressibility factor Z and the cubic residual. Green circles indicate the three analytical roots corresponding to liquid ($Z_L = 0.035$), unstable ($Z_{unstable} = 0.128$), and vapor ($Z_V = 0.815$) phases. (b) Distribution of roots obtained by conditioning each model on residual = 0. Both methods successfully identify all three roots, with the GMM producing slightly sharper distributions due to its analytical conditioning, while FM generates samples through numerical integration of the learned velocity field.

Table 1: Comparison between GMM estimations and analytical solution of Peng-Robinson equation of state example.

Root	GMM estimate	Analytical
Z_{liquid}	0.035 ± 0.001	0.0348
Z_{unstable}	0.128 ± 0.010	0.1280
Z_{vapor}	0.814 ± 0.002	0.8146

The generative approach naturally obtains all three roots simultaneously, whereas iterative methods like Newton-Raphson find only the root nearest to the initial guess while requiring multiple, different initial estimates to find all roots. The generative framework extends naturally to finding complex roots by treating real and imaginary parts as separate dimensions. An example of such a situation is available in the supporting repository (`00ab_complex_roots.ipynb`).

3.3 Unconstrained optimization with generative models

Reactive systems are ubiquitous in chemical engineering applications, and optimizing them is an essential task. The following example considers the maximization of a desired generic product B, which is generated within a reaction network described by consecutive first-order reactions in a batch reactor:

$$\mathrm{A} \xrightarrow{k_1} \mathrm{B} \xrightarrow{k_2} \mathrm{C} \tag{16}$$

where B is the desired product and C is an unwanted byproduct. The following parameters were used: $k_1 = 0.5$ min^{-1} (A $\rightarrow$ B), $k_2 = 0.2$ min^{-1} (B $\rightarrow$ C), $C_{A0} = 1.0$ mol/L. This leads to an optimal solution of $\tau^* \approx 3.05$ min and $C_B(\tau^*) \approx 0.543$ mol/L if the problem is to be solved by the means of classic mathematical programming.

The concentration of intermediate B at time τ is given by Eq. 17.

$$C_B(\tau) = \frac{C_{A0} k_1}{k_2 - k_1} \left(e^{-k_1 \tau} - e^{-k_2 \tau} \right) \tag{17}$$

where C_{A0} is the initial concentration of A, and k_1, k_2 are the rate constants.

The optimization problem is to find the reaction time τ^* that maximizes the concentration of intermediate B:

$$\max_{\tau} C_B(\tau) \tag{18}$$

The first-order necessary condition for a maximum is given by Eq. 19,

$$\frac{dC_B}{d\tau} = C_{A0}k_1\frac{k_2e^{-k_2\tau} - k_1e^{-k_1\tau}}{k_2 - k_1} = 0 \tag{19}$$

And solving Eq. 19 yields the analytical optimum, as illustrated in Eq. 20.

$$\tau^* = \frac{\ln(k_1/k_2)}{k_1 - k_2} \tag{20}$$

The second derivative test confirms this corresponds to a maximum, as shown in Eq. 21

$$\left.\frac{d^2C_B}{d\tau^2}\right|_{\tau=\tau^*} < 0 \tag{21}$$

Rather than solving the optimality condition analytically, the generative approach starts by generating training data consisting of tuples $\left(\tau, C_B(\tau), \frac{dC_B}{d\tau}\right)$. The GMM learns the full joint distribution $p\left(\tau, C_B, \frac{dC_B}{d\tau}\right)$, while the flow matching model is trained directly on the conditional $p\left(\tau \mid \frac{dC_B}{d\tau}\right)$. In both cases the optimum is then identified by sampling from $p\left(\tau \mid \frac{dC_B}{d\tau} = 0\right)$ (Figure 5). Furthermore, it is important to note that this probabilistic inference approach theoretically generalizes to any complex reaction networks where analytical solutions do not exist, including more complex applications in which a first principles model is not readily available.

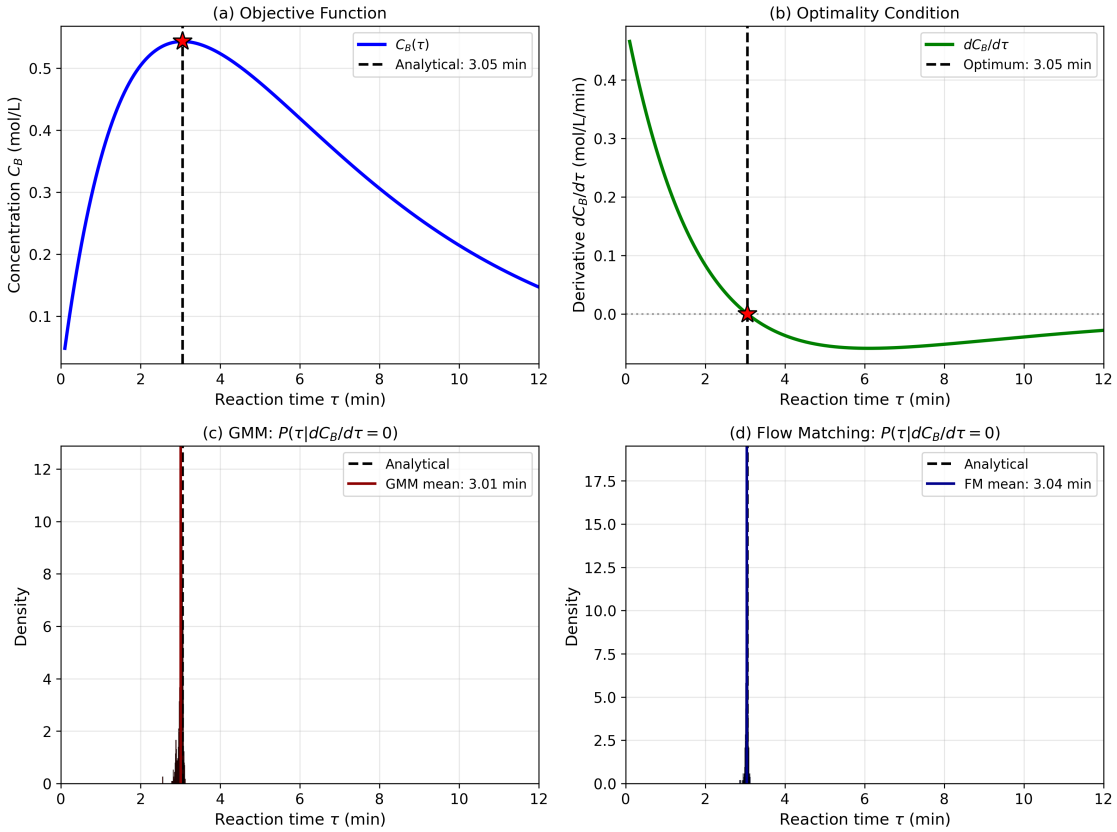


Figure 5: Optimization via conditional sampling for consecutive reactions A $\rightarrow$ B $\rightarrow$ C. (a) Intermediate product concentration $C_B(\tau)$ shows maximum at $\tau^* \approx 3.05$ min. (b) First derivative $dC_B/d\tau$ crosses zero at the optimum. (c-d) GMM and flow matching identify the optimum by sampling from $P(\tau \mid dC_B/d\tau = 0)$, producing narrow distributions (dark red/dark blue solid lines show means) close to the analytical solution (black dashed line): the GMM gives $\tau = 3.01 \pm 0.06$ min and flow matching gives $\tau = 3.04 \pm 0.03$ min, compared to $\tau^* = 3.054$ min. Both models were trained on $\tau \in [0.1, 12]$ min. Both methods solve the optimization problem through probabilistic inference, demonstrating that conditioning on the optimality condition (derivative = 0) yields the optimal reaction time with inherent uncertainty quantification.

3.4 Optimization with equality constraints

If equality constraints are present in the engineering application at hand (which is not uncommon), it is not possible to solve the underlying optimization problem by just conditioning on the objective function's derivative being zero. For example, if a constraint is active, the derivative of the objective function is not equal to zero at the constrained optimal solution. To solve this generatively, we turn to optimization theory and employ the Lagrangian ap-

proach, that reformulates the constrained problem as an unconstrained one, in which the derivatives of the Lagrangian are equal to zero. We can construct this function, generate samples of the inputs (including the Lagrange multiplier λ values) and the derivatives of the Lagrangian. Subsequently, we can obtain solutions by conditioning on $\nabla\mathcal{L} = 0$.

We illustrate this with a practical chemical engineering problem: finding a gasoline blend closest to a target recipe while meeting an octane specification. Assuming an oil refinery blends three components (reformate, isomerate, and alkylate) and wants to minimize deviation from a preferred blend composition $(x_1^*, x_2^*, x_3^*) = (0.4, 0.4, 0.2)$ while satisfying quality constraints. The problem is formulated as shown in Eq. 22.

$$\begin{aligned} \min_{x_1,x_2,x_3} \quad & (x_1 - 0.4)^2 + (x_2 - 0.4)^2 + (x_3 - 0.2)^2 \\ \text{subject to} \quad & \text{RON}_1 x_1 + \text{RON}_2 x_2 + \text{RON}_3 x_3 = \text{RON}_{\text{target}} \\ & x_1 + x_2 + x_3 = 1 \end{aligned} \tag{22}$$

where x_i are the volume fractions of each component and RON_i are the Research Octane Numbers (RON). We use representative values: reformate ($\text{RON}_1 = 95$), isomerate ($\text{RON}_2 = 82$), and alkylate ($\text{RON}_3 = 94$), with a target octane of $\text{RON}_{\text{target}} = 87$ (regular gasoline specification). The quadratic objective ensures an interior solution exists where all blend fractions are positive, which is essential for the Lagrangian approach to work correctly.

The Lagrangian for this problem with two equality constraints is:

$$\mathcal{L}(x_1, x_2, x_3, \lambda_1, \lambda_2) = \sum_{i=1}^{3} (x_i - x_i^*)^2 + \lambda_1(g_1) + \lambda_2(g_2) \tag{23}$$

where $g_1 = \text{RON}_1 x_1 + \text{RON}_2 x_2 + \text{RON}_3 x_3 - \text{RON}_{\text{target}}$ is the octane constraint and $g_2 = x_1 + x_2 + x_3 - 1$ is the mass balance constraint.

The first-order stationarity conditions (the Karush-Kuhn-Tucker conditions, which for equality constraints reduce to the classical Lagrange conditions) require all five partial derivatives of the Lagrangian to equal zero, as shown in Eq. 24.

$$
\begin{aligned}
\frac{\partial \mathcal{L}}{\partial x_1} &= 2(x_1 - 0.4) + 95\lambda_1 + \lambda_2 = 0 \\
\frac{\partial \mathcal{L}}{\partial x_2} &= 2(x_2 - 0.4) + 82\lambda_1 + \lambda_2 = 0 \\
\frac{\partial \mathcal{L}}{\partial x_3} &= 2(x_3 - 0.2) + 94\lambda_1 + \lambda_2 = 0 \\
\frac{\partial \mathcal{L}}{\partial \lambda_1} &= g_1 = 0 \\
\frac{\partial \mathcal{L}}{\partial \lambda_2} &= g_2 = 0
\end{aligned}
\tag{24}
$$

Our generative optimization approach reformulates this constrained optimization, translating it into a sampling problem. We generate samples of $(x_1, x_2, x_3, \lambda_1, \lambda_2)$ along with their corresponding Lagrangian derivatives (which are computed using automatic differentiation (AD)[45]), train a GMM on this joint distribution, and then condition on all five derivatives being zero to recover the optimal blend.

The reference solution from `scipy.optimize.minimize` with the Sequential Least Squares Programming (SLSQP) method yields $x_1 = 0.284$, $x_2 = 0.607$, $x_3 = 0.109$. Conditioning the GMM on $\nabla\mathcal{L} = 0$ recovers this solution with high accuracy: GMM estimates of $x_1 = 0.284 \pm 0.004$, $x_2 = 0.607 \pm 0.008$, $x_3 = 0.109 \pm 0.003$. Figure 6 shows that GMM, flow matching and `scipy` all show good agreement. The optimal blend deviates from the target to satisfy the octane constraint, using more isomerate (which has the lowest octane) and less reformate than the target recipe.

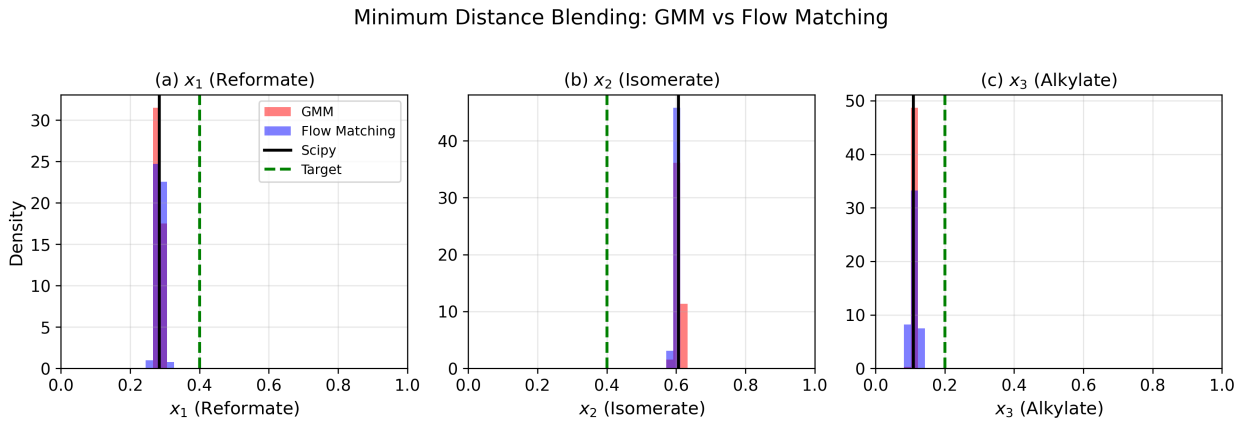


Figure 6: Comparison of GMM and Flow Matching for equality-constrained optimization of a gasoline blending problem. The objective is to find the blend composition closest to a target recipe ($x_1^* = 0.4$, $x_2^* = 0.4$, $x_3^* = 0.2$) while satisfying an octane specification (RON = 87) and mass balance constraint. Panels (a)-(c) show the sample distributions for each blend fraction obtained by conditioning on the KKT conditions ($\nabla\mathcal{L} = 0$). Red histograms: GMM samples; blue histograms: Flow Matching samples; solid black lines: `scipy` SLSQP reference solution; dashed green lines: target blend values. Both generative methods produce distributions centered near the optimal solution, with GMM showing slightly tighter distributions than Flow Matching. The optimal blend deviates from the target to satisfy the octane constraint, using more isomerate (x_2) and less reformate (x_1) than targeted.

3.5 Constrained optimization with an inequality

Inequality constraints bring a new challenge when using a generative sampling-based method as the one proposed in this work: there could be a range of possible values that could satisfy the solution (that is the nature of inequality), hence, it is not possible to condition on one set of values like it is with the equality constraint. A benefit of the conventional mathematical programming approaches such as the implementations in the `minimize` function in `scipy` is one simply changes the constraint type and the optimization formulations.

We cannot use the Lagrange method for an inequality constrained problem as done in the previous example for an equality constrained case. Instead, we consider the barrier method.[46] Recalling the barrier function for the inequality-constrained optimization problem defined as in Eq. 25, in which the objective function incorporates the set of inequality constraints $\mathcal{I}$.

$$f(x) - \mu \sum_{i \in \mathcal{I}} \log c_i(x) \tag{25}$$

From classic nonlinear optimization theory, it is known that when $\mu \to 0$, that is, there is a decreasing sequence of barrier parameter values,[5] the barrier terms vanish and the adapted problem in Eq. 25 collapses into the original inequality problem. Now, the main task is to specify a μ value here and that is usually selected iteratively. We do not do that here, but use a value known to be adequate for our proposed approach. It should be possible to iteratively try different values of μ to convergence, but we rather focus on demonstrating that it is possible to use a generative approach for inequality-constrained optimization problems, as a rich body of literature in inequality constrained optimization already discusses challenges in this area.[5,46]

A physically motivated inequality constraint example arises in chemical reaction equilibrium. Consider the reversible reaction A + B $\rightleftharpoons$ 2C in a batch reactor. Given initial concentrations $C_{A0} = C_{B0} = 1$ mol/L and $C_{C0} = 0$, the concentrations evolve with extent of reaction ξ as in Eq. 26.

$$C_A = 1 - \xi, \quad C_B = 1 - \xi, \quad C_C = 2\xi \tag{26}$$

From the equilibrium condition, Eq. 27 can be derived, and the equilibrium residual (Eq. 28) is obtained.

$$K_{eq} = \frac{C_C^2}{C_A \cdot C_B} = \frac{(2\xi)^2}{(1-\xi)^2} \tag{27}$$

$$f(\xi) = K_{eq}(1-\xi)^2 - 4\xi^2 \tag{28}$$

For $K_{eq} = 1$, Eq. 28 is the quadratic $f(\xi) = 1 - 2\xi - 3\xi^2$, whose roots are $\xi = 1/3$ and $\xi = -1$. Only the first is physically meaningful, since concentrations must be non-

negative, which requires $0 < \xi < 1$. The presence of a second, infeasible root is precisely what motivates an inequality-constrained formulation here: an unconstrained solver started poorly can converge to $\xi = -1$, which corresponds to negative concentrations of C.

We formulate this as an optimization problem: minimize the squared residual $f(\xi)^2$ subject to the inequality constraints $\xi > 0$ and $\xi < 1$. Using the barrier method, we define the augmented objective:

$$\phi(\xi) = f(\xi)^2 - \mu \log(\xi) - \mu \log(1 - \xi) \tag{29}$$

where $\mu = 0.001$ is the barrier parameter. The logarithmic barrier terms become large as ξ approaches the constraint boundaries, keeping solutions in the feasible interior. At the optimum, $\frac{d\phi}{d\xi} = 0$.

To solve this with a generative approach, we sample ξ values in the feasible region $(0, 1)$, compute the barrier objective gradient $\frac{d\phi}{d\xi}$ at each sample using automatic differentiation,[45] train generative models on the joint distribution $[\xi, \frac{d\phi}{d\xi}]$, and condition on the gradient being zero to find the optimal extent. Both GMM and flow matching are capable of successfully identifying $\xi \approx 1/3$, as shown in Figure 7. Note that a finite μ biases the barrier optimum slightly away from the true root; here the minimizer of ϕ is $\xi = 0.33338$ compared to the exact value of $1/3$, a difference of 4.7×10^{-5} that is far smaller than the spread of the sampled distributions.

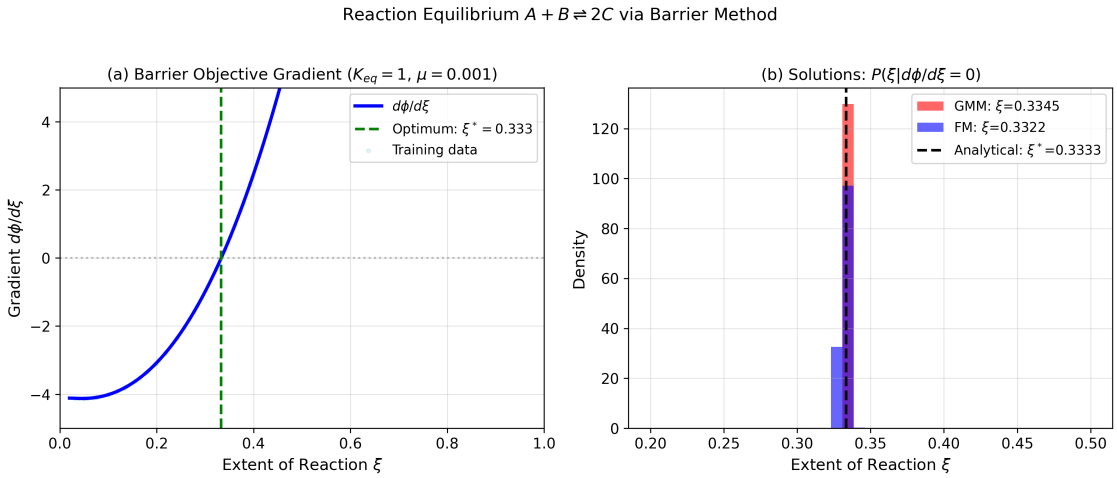


Figure 7: Reaction equilibrium A + B $\rightleftharpoons$ 2C solved via barrier method optimization. (a) The gradient of the barrier objective $\frac{d\phi}{d\xi}$, which equals zero at the equilibrium extent $\xi^* = 1/3$. Training data points shown in light blue. (b) Distributions of ξ values obtained by conditioning GMM (red) and Flow Matching (blue) on $\frac{d\phi}{d\xi} = 0$. Both methods successfully identify the analytical solution (black dashed line).

This example demonstrates that inequality-constrained optimization problems can be solved with the generative approach using the barrier method. By incorporating logarithmic barrier terms and conditioning on the gradient of the augmented objective being zero, we find physically meaningful solutions while respecting the inequality constraints. This approach can also be generalized to more complex reaction networks and constraint sets.

3.6 Parameter estimation with generative models

Parameter estimation in regression is a class of inverse problem.[47] Usually we consider this in the context of optimization as finding a set of parameters (inputs, or decision variables) that minimize an objective function (outputs) of the errors between a model and observations. To recast this idea for a generative model, we consider the dataset as having columns of parameters (inputs) and observations (outputs). The observations could be derivatives of the objective function, and the remaining task is, once again, to condition on those being zero as previously described. We take an alternative approach here, and consider the observations to be some output of a model. Hence, given an observation, we condition on that to obtain

the parameters most consistent with that observation. This approach is fundamentally different from regression where parameters are estimated by minimizing some function of pooled errors.

For a concrete example, we use a series reaction: A $\rightarrow$ B $\rightarrow$ C in a plug flow reactor. The observations we have will be the exit molar flow rates of A and B. We use an inlet molar flow, fixed volume and volumetric flow rate. The system is governed by the two differential equations in Eq. 30.

$$\frac{dF_A}{dV} = -\frac{k_1 F_A}{\nu} \tag{30a}$$

$$\frac{dF_B}{dV} = \frac{k_1 F_A}{\nu} - \frac{k_2 F_B}{\nu} \tag{30b}$$

We sample a range of rate constants and compute the exit molar flows of A and B. This data set is what we will use to build a generative model next. We generate samples of k_1, k_2 in the range of $[0.1, 3]$. We can see what this data looks like in Figure 8.

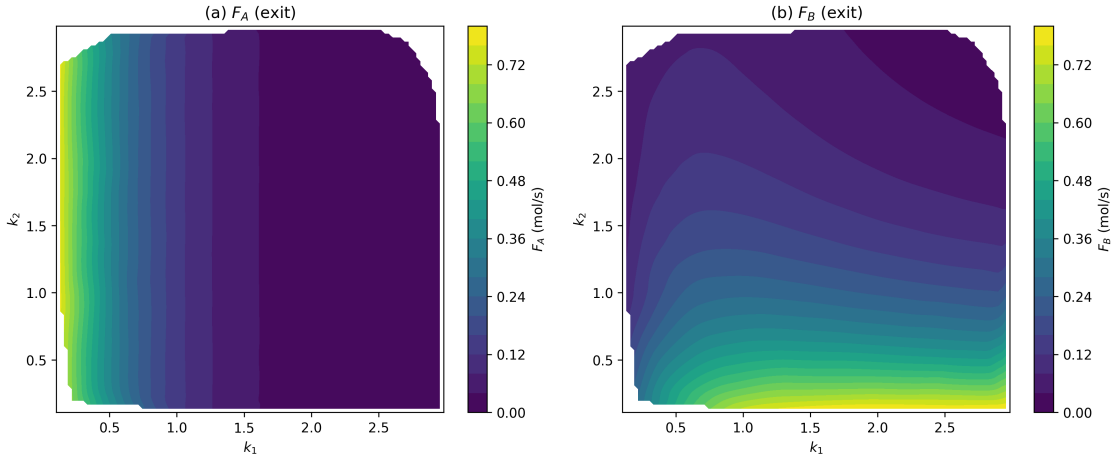


Figure 8: Contour plots of the relationships between k_1 and k_2 and a) F_A and b) F_B.

It is evidently not enough to specify one of these values to uniquely specify the rate constants. In F_A, there are near vertical isolines, and in F_B other isolines. For example, for $k_1 = 2$, F_A will be between 0–0.02 for a broad range of k_2 values, and which value of k_2 that

is chosen depends on the value of F_B that is observed. If you see low F_B, k_2 will be closer to 2, and if F_B is high, then k_2 is closer to 0.2.

For generative parameter estimation we can consider the joint probability distribution that encompasses the variables of interest, that is, $[k_1, k_2, F_A, F_B]$, and then condition the distribution on observed output concentrations to generate the set of parameters that are consistent with those observations. For illustrative purposes we estimate parameters for two scenarios at the reactor outlet:

1. A low molar flow of A with low molar flow of B, i.e. both reactions are fast.

2. A low molar flow of A with high molar flow of B, i.e. reaction 1 is fast, and reaction 2 is slow.

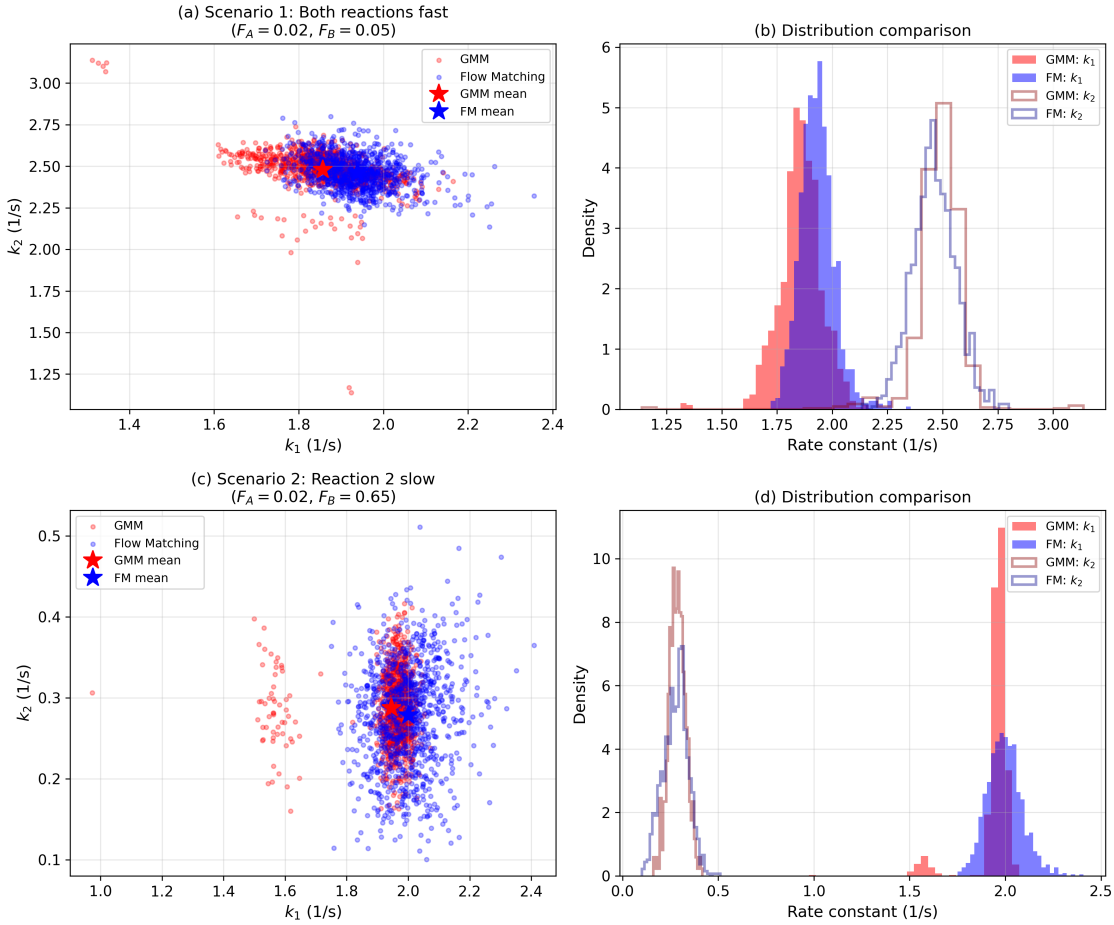


Figure 9: Estimating rate constants (k_1, k_2) from reactor exit flows (F_A, F_B) using GMM and Flow Matching. Left panels show joint distributions in parameter space; right panels show marginal distributions. (a-b) Scenario 1 ($F_A = 0.02$, $F_B = 0.05$): Both reactions fast, requiring high k_1 and k_2. (c-d) Scenario 2 ($F_A = 0.02$, $F_B = 0.65$): First reaction fast, second slow, requiring high k_1 and low k_2. Both methods correctly identify the parameter regimes. GMM shows minor multimodality due to the non-unique inverse problem, while Flow Matching produces smoother distributions. Mean estimates (stars) accurately reproduce the target observations.

It is also possible to estimate a kind of uncertainty. The distribution of the estimated rate constants is due to the surrogate models not fitting the data perfectly well. It is also possible to use this approach to map uncertainties in the inputs to the outputs and vice versa. For example, if one has the confidence intervals of a parameter they can map that range onto outputs with this approach. We illustrate this kind of mapping in Sec. 3.7, and previously showed an approach using inverse mapping concepts.[48]

3.7 Mapping input and output spaces

We previously developed a mapping approach based on differential equations, path integration, and the implicit function theorem.[49] We illustrate here that this can also be achieved by a generative model. The goal is to map a desired output space in the concentration of A, B from a CSTR to the input space of the radius of a CSTR, and the temperature (in units of RT) that achieves it. We have a design with fixed height, and we can modify the radius and temperature (via RT). There are two competing reactions that consume A: A $\rightarrow$ B, with rate $r_1 = k_1 C_A^2$ (second order in A), and A $\rightarrow$ C, with rate $r_2 = k_2 C_A$ (first order in A). B is the desired product and C is an undesired byproduct. We seek a design space with desired amounts of A and B in the exit. We choose a circular region in the output space, and map it to the input space.

Given a radius, we have $V = \pi R^2 H$ and given an RT we have these rate constants for the two forward reactions: $k_1 = \exp(-3/RT)$, $k_2 = \exp(-10/RT)$. Since both reactions consume A, the net species rates are $r_A = -(k_1 C_A^2 + k_2 C_A)$ and $r_B = k_1 C_A^2$. We start with a mole balance on A:

$$0 = F_{A0} - F_A + r_A V \tag{31}$$

Substituting $F_A = \nu_0 C_A$ and r_A, some rearrangement leads to:

$$0 = V k_1 C_A^2 + (\nu_0 + V k_2) C_A - F_{A0} \tag{32}$$

which we can solve with the quadratic equation to get C_A as a function of (R, RT). B is not consumed by either reaction, so the mole balance on B, $0 = -\nu_0 C_B + r_B V$, gives

$$C_B = k_1 C_A^2 V / \nu_0 \tag{33}$$

Thus, we can analytically map the input space to the output space as shown in Figure 10.

We do this by brute force sampling here and show how a circular region in the output space maps to the input space. We choose this example because it is easy to do this analytically, but this forward mapping can also be done in nonlinear programs.

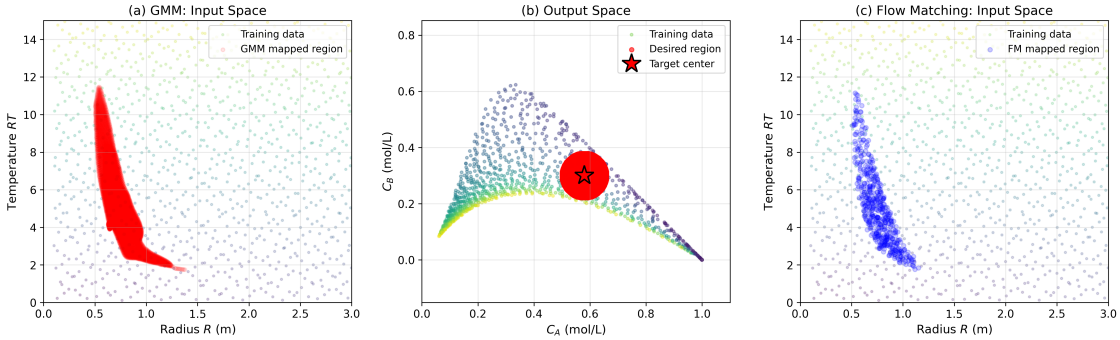


Figure 10: Mapping a desired concentration region (b, red circle: $C_A = 0.58$, $C_B = 0.30 \pm 0.08$ mol/L) to feasible design parameters using (a) GMM and (c) Flow Matching. Both methods identify the irregular, non-convex region in (R, RT) space that produces the target concentrations. Background points show training data colored by distance from origin. The complex shape of the mapped region reveals the nonlinear relationship between reactor size, temperature, and exit concentrations, demonstrating that multiple design configurations can achieve the same production target. Generative models provide this inverse mapping directly by conditioning the learned joint distribution, without requiring the differential-equation formulation of our earlier approach.[49]

This is not the same as learning a forward or inverse model, e.g. a neural network or Gaussian process model. Instead, it is an approximate, data-driven mapping through a joint distribution model. It is not as smooth as the mapping by ODEs,[49] or an analytical mapping, but it might be suitable for some applications, such as when sensor data is available but a mechanistic model is not fully developed or is computationally expensive to evaluate in the inverse mapping direction. Furthermore, building the generative model from more samples would improve the mapping accuracy. It is also possible to map in the forward direction, e.g. from a desired input space to an output space. This has applications in uncertainty propagation,[48] for instance.

There is a connection between the mapping of input and output spaces by differential equations we previously developed[49] and the generative models we use here. Previously[49] we used a differentiable model to derive a set of ODEs that connect the input and output spaces.

We considered a point-wise mapping where one could use the ODEs to transform a single point in one space to the other space. In flow-mapping one uses machine learning to learn the vector field that defines the ODEs that transform the input distribution to the output distribution. Then, by sampling from one space it is possible to transform that sample into the other space. In this way, we might view these models as data-driven versions of the model-driven mapping idea we developed in our previous work.[48,49]

3.8 Process flowsheet constrained optimization and feasibility

As a capstone example illustrating a more complex, higher dimensional, nonlinear process we consider a flowsheet constrained optimization. More often than not, sensor data and/or plant measurements are recorded and stored in data historians, being particularly useful within distributed control system (DCS) architectures which are implemented seeking several objectives such as plant-wide process control, fault diagnosis, hazardous and operability studies (HAZOP), real time optimization and process monitoring. This ever-increasing amount of data provides a clear opportunity to train generative models that can be used to pre-screen operating regions, including optimality driven ones. Such generative models can be used in parallel to the development of rigorous, first-principles-based process models, for instance. In summary, if process data is available, it is possible to use it to build generative joint probability distributions of objective functions that can represent operating cost/profit of a given process, and process variables/constraints that may need to be imposed. Similar approaches have been developed in the context of reinforcement learning for bioprocess optimization,[50] using policy gradients.

It is important to note that the constraints of a given process would be typically inherently represented in the historian data itself, since control structures and operators simultaneously and continuously maintain the process within the feasible region whenever possible. Furthermore, even if the historian data contains points outside the feasible region of operation, this data is equally important as they can bound the training data regime, reducing the

possibility of extrapolation.

As an example of a process flowsheet, we consider an evaporation process that has been used extensively in the plant-wide control literature[51,52] as a benchmark of self-optimizing control[53,54] algorithms. The evaporation process is illustrated in Figure 11. The main goal of this process is to increase the concentration of some generic dilute liquor through the evaporation of a solvent from the feed, employing a heat exchanger as depicted. The description of each process variable is available in Table 2, based on the original source.[51]

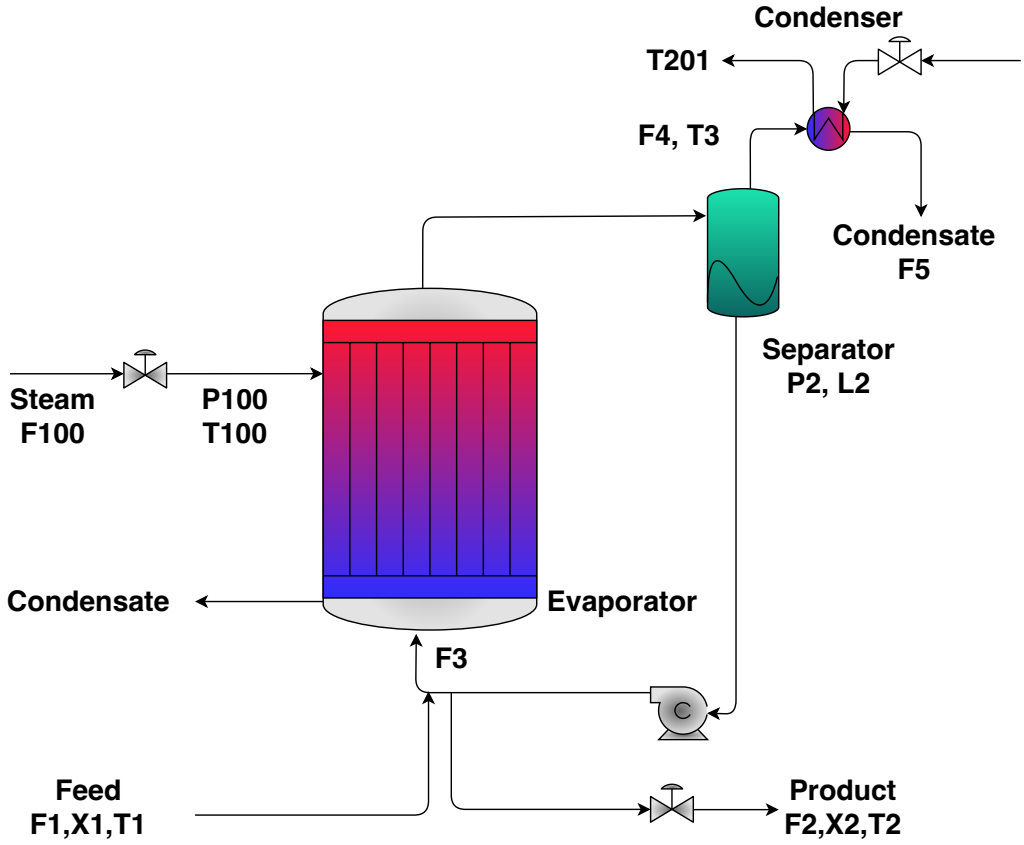


Figure 11: Schematic of evaporation process.

Table 2: Process variables and descriptions

Variable	Description	Variable	Description
F_1	feed flow rate	L_2	separator level
F_2	product flow rate	P_2	operating pressure
F_3	circulating flow rate	F_{100}	steam flow rate
F_4	vapor flow rate	T_{100}	steam temperature
F_5	condensate flow rate	P_{100}	steam pressure
X_1	feed composition	Q_{100}	heat duty
X_2	product composition	F_{200}	cooling water flow rate
T_1	feed temperature	T_{200}	inlet temperature of cooling water
T_2	product temperature	T_{201}	outlet temperature of cooling water
T_3	vapor temperature	Q_{200}	condenser duty

We follow the original source of the problem definition,[51] in which $X_1 = 5\%$, $T_1 = 40\,^{\circ}\mathrm{C}$ and $T_{200} = 25\,^{\circ}\mathrm{C}$ are considered process disturbances at their nominal values. Furthermore, the process flow rates are in kg/min, temperatures in °C, pressures in kPa, heat duties in kW and compositions in percent. The constrained optimization problem maximizes the operational profit of this process as shown previously,[51] subject to several process constraints. We write it in Eq. 34 in the equivalent minimization form, so the reported objective values are negative profits. This leads to the optimization problem described in Eq. 34, to be solved at steady-state.

$$\min \quad 600F_{100} + 0.6F_{200} + 1.009\,(F_2 + F_3) + 0.2F_1 - 4800F_2 \tag{34a}$$

$$\text{s.t.} \quad \frac{\mathrm{d}L_2}{\mathrm{d}t} = \frac{F_1 - F_4 - F_2}{20} = 0 \tag{34b}$$

$$\frac{\mathrm{d}X_2}{\mathrm{d}t} = \frac{F_1X_1 - F_2X_2}{20} = 0 \tag{34c}$$

$$\frac{\mathrm{d}P_2}{\mathrm{d}t} = \frac{F_4 - F_5}{4} = 0 \tag{34d}$$

$$F_4 = \frac{Q_{100} - 0.07F_1\,(T_2 - T_1)}{38.5} \tag{34e}$$

$$T_2 = 0.5616P_2 + 0.3126X_2 + 48.43 \tag{34f}$$

$$T_3 = 0.507P_2 + 55.0 \tag{34g}$$

$$T_{100} = 0.1538P_{100} + 90.0 \tag{34h}$$

$$Q_{100} = 0.16\,(F_1 + F_3)\,(T_{100} - T_2) \tag{34i}$$

$$F_{100} = \frac{Q_{100}}{36.6} \tag{34j}$$

$$Q_{200} = \frac{0.9576F_{200}\,(T_3 - T_{200})}{0.14F_{200} + 6.84} \tag{34k}$$

$$T_{201} = T_{200} + \frac{13.68\,(T_3 - T_{200})}{0.14F_{200} + 6.84} \tag{34l}$$

$$F_5 = \frac{Q_{200}}{38.5} \tag{34m}$$

$$X_2 \geq 35.5\% \tag{34n}$$

$$40 \text{ kPa} \leq P_2 \leq 80 \text{ kPa} \tag{34o}$$

$$P_{100} \leq 400 \text{ kPa} \tag{34p}$$

$$0 \leq F_{200} \leq 400 \text{ kg/min} \tag{34q}$$

$$0 \leq F_1 \leq 20 \text{ kg/min} \tag{34r}$$

$$0 \leq F_3 \leq 100 \text{ kg/min} \tag{34s}$$

The objective function comprises terms containing operating costs of water, steam and pumping,[51] while the fourth and last terms correspond to raw material cost and product

revenue, respectively. It is known[51,52] that under nominal disturbances, the optimal operation is constrained at $X_2 = 35.5\%$ and $P_{100} = 400$ kPa, with an optimal economic operation of \$582.233/h profit, i.e. an objective value of -582.233 \$/h in the minimization form of Eq. 34.

We employ our proposed approach by generating synthetic data corrupted with pseudo-random noise, in order to mimic the presence of noise in real historian data, and we build a generative model of the joint probability distribution of the entire optimization space. This corresponds to using sensor data from the process measurements to evaluate the objective function and process constraints defined as in Eq. 34.

Differently from the reaction equilibrium example, where we used the barrier method, in this case-study a *constraint bound and residual conditioning* approach is employed, that leverages the structure of inequality-constrained optimization problems. The main idea is that when inequality constraints are active at optimality, they effectively become equality constraints. Since it is known from the "product giveaway rule" [55–57] that compositions will be nominally active constraints, as well as inventories and pressures as discussed in the literature,[57] one is able to evaluate the feasible region at the bound of such variables. Hence, the constraints related to product specification and operating pressure at $X_2 = 35.5\%$ and $P_{100} = 400$ kPa are treated at their respective bounds as conditioning variables.

To generate diverse training data, we adopt a *constraint variation* approach: we solve a series of evaporator problems over a grid of constraint configurations ($X_2^{\min} \in [33, 38]\%$, $P_{100}^{\max} \in [260, 400]$ kPa), in which some operational values are feasible with respect to the optimization problem in Eq. 34 while operation outside the range of X_2 and P_{100} are not (Figure 12). Each configuration yields different optimal and suboptimal solutions where the inequality constraints are active at their respective bounds, providing a diverse training dataset from an operational standpoint. This diversity is essential for the generative model to learn how optimal solutions vary with constraint bounds. We argue that a real chemical plant operates similarly, in which controllers and operators are maintaining the plant at the

feasible region but extracting the maximum operation that can be possibly done.

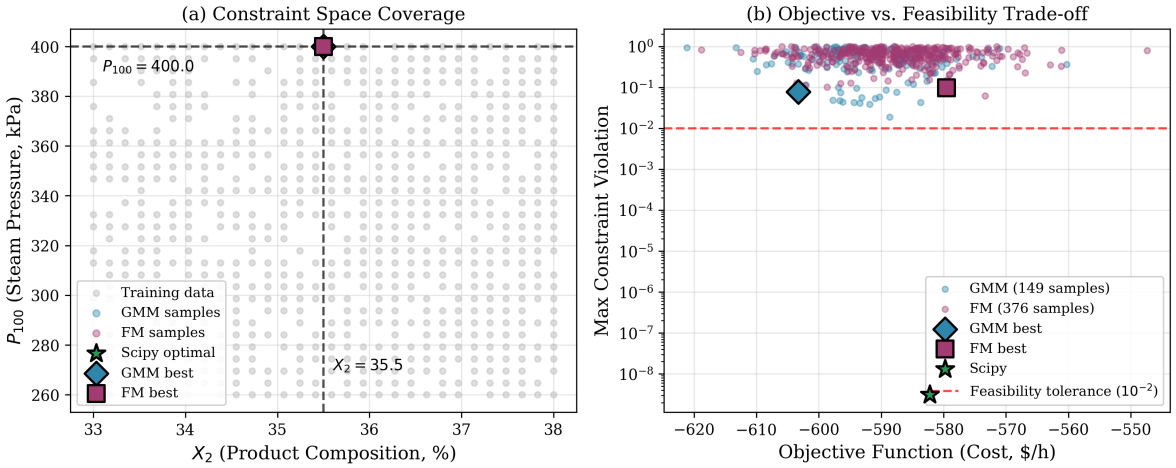


Figure 12: Training data coverage and generative model sampling for the evaporator optimization problem. (a) Constraint space showing the diverse training data generated via constraint variation ($X_2^{\min} \in [33, 38]\%$, $P_{100}^{\max} \in [260, 400]$ kPa), along with samples generated by GMM and Flow Matching when conditioned on the target values $X_2 = 35.5\%$ and $P_{100} = 400$ kPa. The GMM samples cluster tightly at the conditioned values, while Flow Matching samples show slight spread around the target. (b) Trade-off between objective function value and constraint satisfaction. Both methods generate samples near the `scipy` optimal solution, with Flow Matching producing more samples in the near-feasible region (violation < 0.1).

Lastly, our proposed approach learns the joint probability distribution $p(\boldsymbol{x}, \boldsymbol{h}(\boldsymbol{x}))$, where $\boldsymbol{h}(\boldsymbol{x}) \in \mathbb{R}^{12}$ represents the equality constraint residuals (the process model equations). At inference time, we condition on three types of information: (1) $X_2 = 35.5\%$, (2) $P_{100} = 400$ kPa, and (3) $\boldsymbol{h}(\boldsymbol{x}) = \mathbf{0}$. This yields 14 conditioning dimensions total (2 constraint bounds + 12 constraint residuals). The approach is conceptually similar to parameter estimation: the constraint bounds serve as "parameters" that vary across training examples but are fixed to specific target values at inference time.

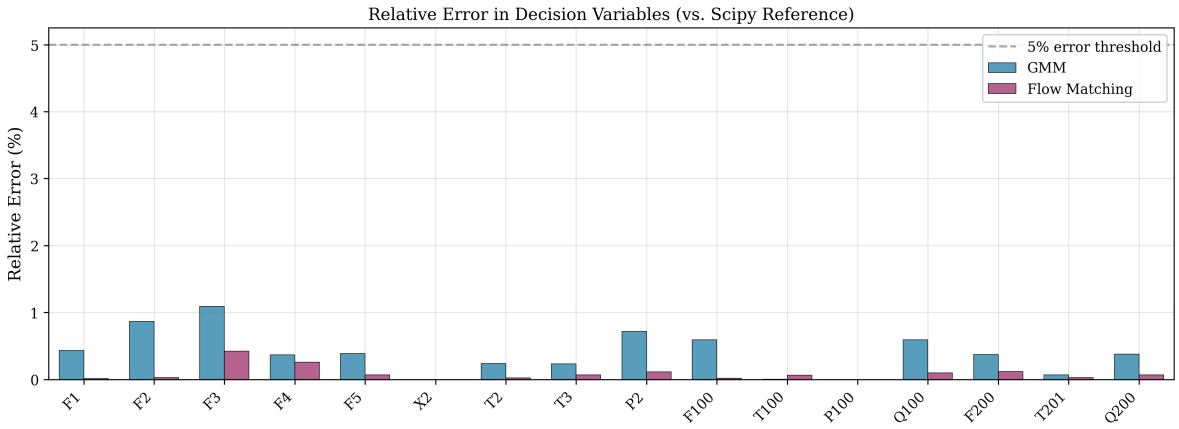


Figure 13: Relative error in each decision variable for the best GMM and Flow Matching solutions compared to the `scipy` reference. Every decision variable is recovered to better than 1.1% relative error by both methods, far below the 5% threshold (dashed line). The conditioning variables X_2 and P_{100} show near-zero error for GMM (since they are directly conditioned) and very small error for Flow Matching. The largest deviation in both cases is in F_3, at 1.09% for GMM and 0.42% for Flow Matching; the remaining deviations in flow rates and duties reflect the sensitivity of these variables to small perturbations in the coupled system of equations.

Figure 13 shows the relative error in each decision variable for the best solutions found by GMM and Flow Matching compared to the `scipy` reference solution. Every decision variable is recovered to better than 1.1% relative error, far below the 5% threshold, indicating that both generative methods recover each individual decision variable closely. The conditioning variables X_2 and P_{100} show near-zero error for GMM (since they are directly conditioned) and very small error for Flow Matching. The largest deviation in both cases is in F_3, at 1.09% for GMM and 0.42% for Flow Matching, and reflects the sensitivity of these variables to small perturbations in the coupled system of equations. Figure 14 summarizes the deviation from the `scipy` reference objective. The GMM solution differs by 3.62% and the flow matching solution by 0.46%, compared with 0.30% for the literature solution. Neither figure should be read as an optimality gap: the corresponding samples carry maximum equality-constraint violations of 7.8×10^{-2} and 9.8×10^{-2}, roughly an order of magnitude above the 10^{-2} tolerance used here to declare a point feasible, and the GMM objective (-603.30 \$/h) falls below the reference (-582.23 \$/h) because the point is infeasible rather than because it is

a better optimum. A purely data-driven approach therefore locates the neighborhood of the optimum from data alone, but the samples it returns are not feasible to the tolerance a conventional solver achieves, and a subsequent projection or local refinement step would be needed to recover a usable operating point.

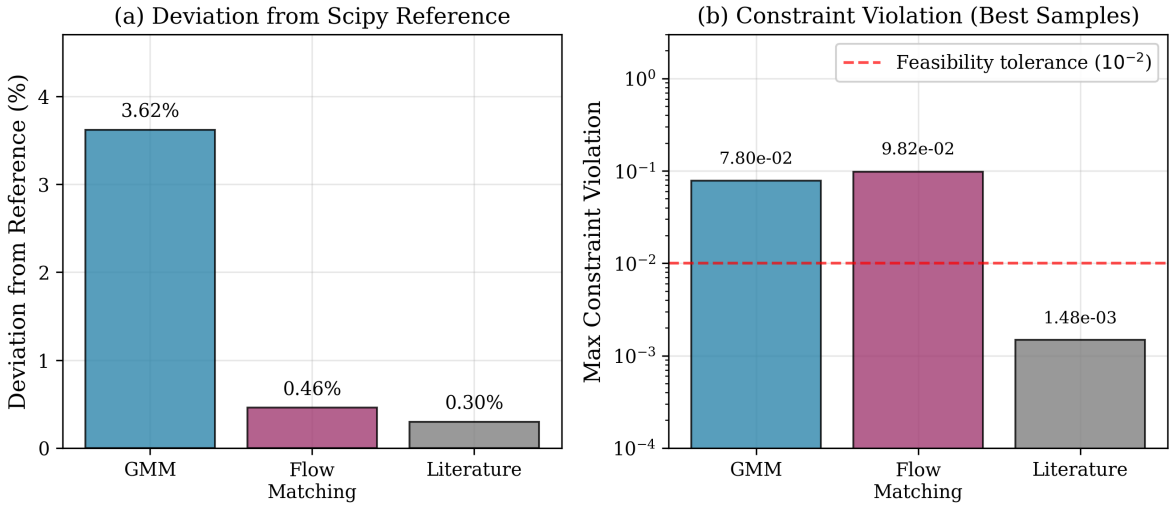


Figure 14: Summary performance comparison for the best solutions found by each generative optimization approach. (a) Deviation from the `scipy` SLSQP reference objective (-582.23 \$/h). The GMM solution differs by 3.62% and flow matching by 0.46%, compared with 0.30% for the literature solution. (b) Maximum equality constraint violation for the best sample from each method, on a logarithmic scale. The generative samples (7.8×10^{-2} for GMM and 9.8×10^{-2} for flow matching) exceed the 10^{-2} feasibility tolerance used in this analysis (dashed line) by roughly an order of magnitude, unlike the literature solution shown alongside them (1.5×10^{-3}) or the `scipy` reference (3.1×10^{-9}). The GMM objective (-603.30 \$/h) therefore lies below the reference as a consequence of this residual infeasibility rather than a better optimum. Conditioning on $X_2 = 35.5\%$, $P_{100} = 400$ kPa, and $\boldsymbol{h}(\boldsymbol{x}) = \mathbf{0}$ guides the models into the neighborhood of the feasible optimum, but does not by itself enforce feasibility.

3.9 Limitations of generative optimization

The generative optimization approach fundamentally relies on the construction of a data-driven surrogate model, and the quality of the solutions relies on the quality of the surrogate model. Like other data-driven surrogate models, generative models are not reliable in extrapolation, or out-of-domain sampling. This can lead to erroneous or physically unrealistic

results. In a related issue, if there are sharp changes in the true joint probability distribution that are not well sampled by the training data, it is unlikely these models "learn" about it, and they may miss these features.

Issues related to dimensionality also need to be addressed. We anticipate that with an increased number of dimensions, numerical instability, curse of dimensionality and training times could become larger problems that were not currently found in the extensive case studies. Furthermore, domain knowledge is still necessary regarding active constraint regions to properly handle inequalities, which is done naturally in mathematical programming formulations. Lastly, the hyperparameter tuning and model selection problems are still present, a reality of any machine learning formulation.

We reiterate that although the idea proposed in this work is novel and has potential to be ubiquitously applied in science and engineering applications, more work needs to be done on edge cases regarding data availability, hyperparameter tuning and generative model selection, for instance.

4 Conclusions

We have illustrated several examples of using generative models in optimization tasks that ranged from root finding, optimization, optimization with constraints, parameter estimation and state mapping. In each case the solution is obtained by conditional generation from a distribution trained from samples of inputs and outputs. For root-finding, the outputs are function values. In optimization, the outputs include derivatives. For constrained optimization, the outputs were derivatives of an augmented Lagrange function. In parameter estimation, we used observations for the outputs. The identification of the input / output pairs that allow one to generate conditional samples from specific values is the key to using generative models this way.

There are a few ways this generative approach could fail. First, if it is hard to model the

underlying joint probability distribution, or learn the transformation the approach would obviously fail. Second, even if it succeeds, there is no guarantee one finds all the possible solutions, even of the surrogate function. The surrogate function may not have sufficient accuracy to find the true solutions of the real system, or may have artifacts that lead to false solutions. The models are built from data and the solutions are found by sampling. For well-behaved broad distributions, sampling should work very well. For poorly-behaved distributions, e.g. where there is a very sharp probability, it is possible to miss a result, both in building the model and sampling it for solutions.

It is evident this is not a panacea solution to optimization. Conventional methods may be more efficient and accurate for some problems, especially when there is a model available. Even regular machine learned models with conventional methods may be better when they have desirable properties, e.g. converting a Rectified Linear Unit (ReLU) neural network to a mixed integer program. It remains unknown if we can find a way to fully include inequality constraints; in this work we relied on an approximate barrier function. Fundamentally, the whole approach relies on a surrogate model, and is limited by that model: the solutions are to the surrogate model, and not the underlying true model. That is a feature when there is no actual model, but rather data. Sampling strategies can be essential to success, and there are not yet established methods to efficiently sample the data one needs. Since this is ultimately a data-driven approach, it is not likely to be accurate in sample generation with conditions that are out of distribution. As with other machine learning models, data-driven generative models should only be expected to be valid for "in-domain" sampling, but in high-dimensional spaces it is not always obvious how to quantify what is in or out of domain. Although one can use sampling to quantify variance in predictions, that variance is a combination of model inadequacy and noise in the data.

Nevertheless, this work shows that generative methods have interesting possibility in a broad range of applications we have not previously considered them for in optimization, as surrogate models, and likely others we have not thought of yet. There are many generative

models in existence today, and new ones are still being found. It seems likely these will mitigate problems we have illustrated in this work, and make new approaches possible in the future. We are most excited by the new way of thinking about problems that this approach now enables.

Acknowledgement

JRK acknowledges Prof. Carl D. Laird for suggesting the barrier function approach for problems with inequality constraints.

This work received no external funding.

Generative AI (Claude Code with the Opus 4.5 model) was used in the preparation of this work, for code development, literature review, and portions of the manuscript text. The nature and extent of that use is described in Sec. 2.3. The authors reviewed all content and take full responsibility for it.

The code and data supporting this work are available in the GitHub repository https://github.com/KitchinHUB/generative-optimization, which contains extensive examples in the form of Jupyter notebooks for the GMM and flow-matching examples, and the notebooks that were used to generate the figures in this manuscript. In addition to the examples discussed in the manuscript there are additional examples on advanced optimization topics and visualization, hyperparameter tuning and learning curves, examples of limitations, the SKILL.md file for use with LLMs and automation tools for running the notebooks.

References

(1) Dantzig, G. B. *A history of scientific computing*; Association for Computing Machinery, 1990; pp 141–151.

(2) Han, S. P. A globally convergent method for nonlinear programming. *Journal of Optimization Theory and Applications* **1977**, *22*, 297–309.

(3) Powell, M. J. D. A fast algorithm for nonlinearly constrained optimization calculations. Numerical Analysis. Berlin, Heidelberg, 1978; pp 144–157.

(4) Wilson, R. B. A Simplicial Algorithm for Concave Programming. Doctor of Business Administration dissertation, Harvard Business School, Boston, MA, 1963.

(5) Wächter, A.; Biegler, L. T. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming* **2006**, *106*, 25–57.

(6) Bonami, P.; Biegler, L. T.; Conn, A. R.; Cornuéjols, G.; Grossmann, I. E.; Laird, C. D.; Lee, J.; Lodi, A.; Margot, F.; Sawaya, N.; Wächter, A. An algorithmic framework for convex mixed integer nonlinear programs. *Discrete Optimization* **2008**, *5*, 186–204, In Memory of George B. Dantzig.

(7) Grossmann, I. E.; Ruiz, J. P. Generalized Disjunctive Programming: A Framework for Formulation and Alternative Algorithms for MINLP Optimization. Mixed Integer Nonlinear Programming. New York, NY, 2012; pp 93–115.

(8) Floudas, C.; Akrotirianakis, I.; Caratzoulas, S.; Meyer, C.; Kallrath, J. Global optimization in the 21st century: Advances and challenges. *Computers & Chemical Engineering* **2005**, *29*, 1185–1202, Selected Papers Presented at the 14th European Symposium on Computer Aided Process Engineering.

(9) Floudas, C. A.; Gounaris, C. E. A review of recent advances in global optimization. *Journal of Global Optimization* **2009**, *45*, 3–38.

(10) Boukouvala, F.; Misener, R.; Floudas, C. A. Global optimization advances in Mixed-

Integer Nonlinear Programming, MINLP, and Constrained Derivative-Free Optimization, CDFO. *European Journal of Operational Research* **2016**, *252*, 701–727.

(11) Virtanen, P. et al. Scipy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* **2020**, *17*, 261–272.

(12) Bynum, M. L.; Hackebeil, G. A.; Hart, W. E.; Laird, C. D.; Nicholson, B. L.; Siirola, J. D.; Watson, J.-P.; Woodruff, D. L. *Pyomo - Optimization Modeling in Python*; Springer Optimization and Its Applications; Springer International Publishing, 2021.

(13) Beal, L.; Hill, D.; Martin, R.; Hedengren, J. Gekko Optimization Suite. *Processes* **2018**, *6*, 106.

(14) Gurobi Optimization, LLC Gurobi Optimizer Reference Manual. 2024; https://www.gurobi.com.

(15) Andersen, E. D.; Andersen, K. D. In *High Performance Optimization*; Frenk, H., Roos, K., Terlaky, T., Zhang, S., Eds.; Springer US: Boston, MA, 2000; pp 197–232.

(16) Dunning, I.; Huchette, J.; Lubin, M. Jump: a Modeling Language for Mathematical Optimization. *SIAM Review* **2017**, *59*, 295–320.

(17) Biegler, L. T.; Grossmann, I. E. Retrospective on optimization. *Computers & Chemical Engineering* **2004**, *28*, 1169–1192.

(18) NEOS Guide Optimization Problem Types. 2025; https://neos-guide.org/guide/types/, Accessed: 2025-04-22.

(19) Harshvardhan, G. M.; Gourisaria, M. K.; Pandey, M.; Rautaray, S. S. A Comprehensive Survey and Analysis of Generative Models in Machine Learning. *Computer Science Review* **2020**, *38*, 100285.

(20) Ardizzone, L.; Kruse, J.; Rother, C.; Köthe, U. Analyzing Inverse Problems with Invertible Neural Networks. International Conference on Learning Representations. 2019.

(21) Dasgupta, A.; Ramaswamy, H.; Murgoitio-Esandi, J.; Foo, K. Y.; Li, R.; Zhou, Q.; Kennedy, B. F.; Oberai, A. A. Conditional Score-Based Diffusion Models for Solving Inverse Elasticity Problems. *Computer Methods in Applied Mechanics and Engineering* **2025**, *433*, 117425.

(22) Kitchin, J. R. Solving an inverse problem with generative models. *Digital Discovery* **2025**, *4*, 1856–1869.

(23) Alcazar, J.; Vakili, M. G.; Kalayci, C. B.; Perdomo-Ortiz, A. Enhancing Combinatorial Optimization With Classical and Quantum Generative Models. *Nature Communications* **2024**, *15*, 2761.

(24) Wyrod, P.; Chattopadhyay, A.; Venturi, D. Generative forecasting with joint probability models. 2025; https://arxiv.org/abs/2512.24446.

(25) Muthyala, M. R.; Sorourifar, F.; Tan, T.; Peng, Y.; Paulson, J. A. Generative Multi-objective Bayesian Optimization with Scalable Batch Evaluations for Sample-Efficient De Novo Molecular Design. *Industrial & Engineering Chemistry Research* **2026**, *65*, 628–642.

(26) Yao, M. S.; Zeng, Y.; Bastani, H.; Gardner, J.; Gee, J. C.; Bastani, O. Generative Adversarial Model-Based Optimization Via Source Critic Regularization. Advances in Neural Information Processing Systems (NeurIPS). 2024.

(27) Becker, J.; Wahle, J. P.; Gipp, B.; Ruas, T. Text Generation: A Systematic Literature Review of Tasks, Evaluation, and Challenges. *Journal of Artificial Intelligence Research* **2026**, *86*.

(28) Elasri, M.; Elharrouss, O.; Al-Maadeed, S.; Tairi, H. Image Generation: a Review. *Neural Processing Letters* **2022**, *54*, 4609–4646.

(29) Waqar, F. G.; Patel, S.; Simon, C. M. A Tutorial on the Bayesian Statistical Approach to Inverse Problems. *APL Machine Learning* **2023**, *1*, 041101.

(30) Nelder, J. A.; Mead, R. A Simplex Method for Function Minimization. *The Computer Journal* **1965**, *7*, 308–313.

(31) Yang, L.; Cheung, N.-M.; Li, J.; Fang, J. Deep Clustering by Gaussian Mixture Variational Autoencoders With Graph Embedding. 2019 IEEE/CVF International Conference on Computer Vision (ICCV). 2019; pp 6439–6448.

(32) Viroli, C.; McLachlan, G. J. Deep Gaussian mixture models. *Statistics and Computing* **2019**, *29*, 43–51.

(33) Bishop, C. M. *Pattern Recognition and Machine Learning*; Information Science and Statistics; Springer: New York, 2006.

(34) Fabisch, A. gmr: Gaussian Mixture Regression. *Journal of Open Source Software* **2021**, *6*, 3054.

(35) Deisenroth, M. P.; Faisal, A. A.; Ong, C. S. *Mathematics for machine learning*; Cambridge University Press, 2020.

(36) Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; Vanderplas, J.; Passos, A.; Cournapeau, D.; Brucher, M.; Perrot, M.; Duchesnay, E. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **2011**, *12*, 2825–2830.

(37) Ghahramani, Z.; Jordan, M. Supervised learning from incomplete data via an EM approach. Advances in Neural Information Processing Systems. 1993.

(38) Lipman, Y.; Chen, R. T. Q.; Ben-Hamu, H.; Nickel, M.; Le, M. Flow Matching for Generative Modeling. International Conference on Learning Representations (ICLR). 2023.

(39) Tong, A.; Fatras, K.; Malkin, N.; Huguet, G.; Zhang, Y.; Rector-Brooks, J.; Wolf, G.; Bengio, Y. Improving and generalizing flow-based generative models with minibatch optimal transport. *Transactions on Machine Learning Research* **2024**, Expert Certification.

(40) Chen, R. T. Q.; Rubanova, Y.; Bettencourt, J.; Duvenaud, D. K. Neural Ordinary Differential Equations. Advances in Neural Information Processing Systems (NeurIPS). 2018.

(41) Kingma, D. P.; Ba, J. Adam: A Method for Stochastic Optimization. International Conference on Learning Representations (ICLR). 2015.

(42) Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; Desmaison, A.; Köpf, A.; Yang, E. Z.; DeVito, Z.; Raison, M.; Tejani, A.; Chilamkurthy, S.; Steiner, B.; Fang, L.; Bai, J.; Chintala, S. PyTorch: An Imperative Style, High-Performance Deep Learning Library. Advances in Neural Information Processing Systems (NeurIPS). 2019.

(43) Song, Y.; Sohl-Dickstein, J.; Kingma, D. P.; Kumar, A.; Ermon, S.; Poole, B. Score-Based Generative Modeling through Stochastic Differential Equations. International Conference on Learning Representations (ICLR). 2021.

(44) Papamakarios, G.; Nalisnick, E.; Rezende, D. J.; Mohamed, S.; Lakshminarayanan, B. Normalizing Flows for Probabilistic Modeling and Inference. *Journal of Machine Learning Research* **2021**, *22*, 1–64.

(45) Bradbury, J.; Frostig, R.; Hawkins, P.; Johnson, M. J.; Leary, C.; Maclaurin, D.; Necula, G.; Paszke, A.; VanderPlas, J.; Wanderman-Milne, S.; Zhang, Q. JAX: composable transformations of Python+NumPy programs. 2018; http://github.com/jax-ml/jax.

(46) Nocedal, J.; Wright, S. *Numerical Optimization*; Springer Series in Operations Research and Financial Engineering; Springer New York, 2006.

(47) Aster, R. C.; Borchers, B.; Thurber, C. H. In *Parameter Estimation and Inverse Problems (Second Edition)*, second edition ed.; Aster, R. C., Borchers, B., Thurber, C. H., Eds.; Academic Press: Boston, 2013; pp 1–23.

(48) Alves, V.; Laird, C. D.; Lima, F. V.; Kitchin, J. R. Mapping Uncertainty Using Differentiable Programming. *AIChE Journal* **2025**, e18940.

(49) Alves, V.; Kitchin, J. R.; Lima, F. V. An inverse mapping approach for process systems engineering using automatic differentiation and the implicit function theorem. *AIChE Journal* **2023**, e18119.

(50) Petsagkourakis, P.; Sandoval, I.; Bradford, E.; Zhang, D.; del Rio-Chanona, E. Reinforcement learning for batch bioprocess optimization. *Computers & Chemical Engineering* **2020**, *133*, 106649.

(51) Kariwala, V.; Cao, Y.; Janardhanan, S. Local Self-Optimizing Control with Average Loss Minimization. *Industrial & Engineering Chemistry Research* **2008**, *47*, 1150–1158.

(52) Alves, V. M. C.; Lima, F. S.; Silva, S. K.; Araujo, A. C. B. Metamodel-Based Numerical Techniques for Self-Optimizing Control. *Industrial & Engineering Chemistry Research* **2018**, *57*, 16817–16840.

(53) Skogestad, S. Plantwide control: the search for the self-optimizing control structure. *Journal of Process Control* **2000**, *10*, 487–507.

(54) Jäschke, J.; Cao, Y.; Kariwala, V. Self-optimizing control - A survey. *Annual Reviews in Control* **2017**, *43*, 199–223.

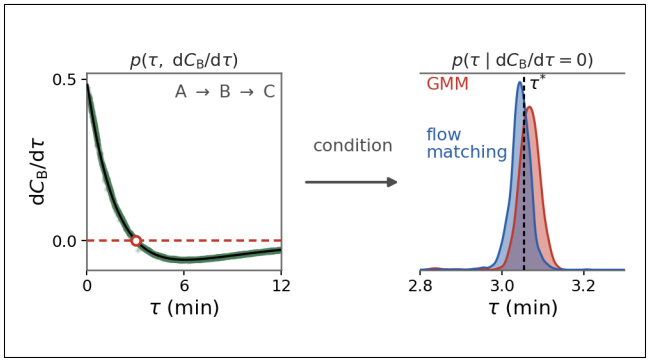
(55) Jacobsen, M. G.; Skogestad, S. Active Constraint Regions for Optimal Operation of

Chemical Processes. *Industrial & Engineering Chemistry Research* **2011**, *50*, 11226–11236.

(56) Jacobsen, M. G.; Skogestad, S. Active Constraint Regions for Optimal Operation of Distillation Columns. *Industrial & Engineering Chemistry Research* **2012**, *51*, 2963–2973.

(57) Minasidis, V.; Skogestad, S.; Kaistha, N. In *12th International Symposium on Process Systems Engineering and 25th European Symposium on Computer Aided Process Engineering*; Gernaey, K. V., Huusom, J. K., Gani, R., Eds.; Computer Aided Chemical Engineering; Elsevier, 2015; Vol. 37; pp 101–108.

TOC Graphic


$p(\tau,\ dC_B/d\tau)$
A → B → C
0.5
0.0
$dC_B/d\tau$
0
6
12
τ (min)
condition
$p(\tau \mid dC_B/d\tau = 0)$
GMM
flow
matching
τ^*
2.8
3.0
3.2
τ (min)